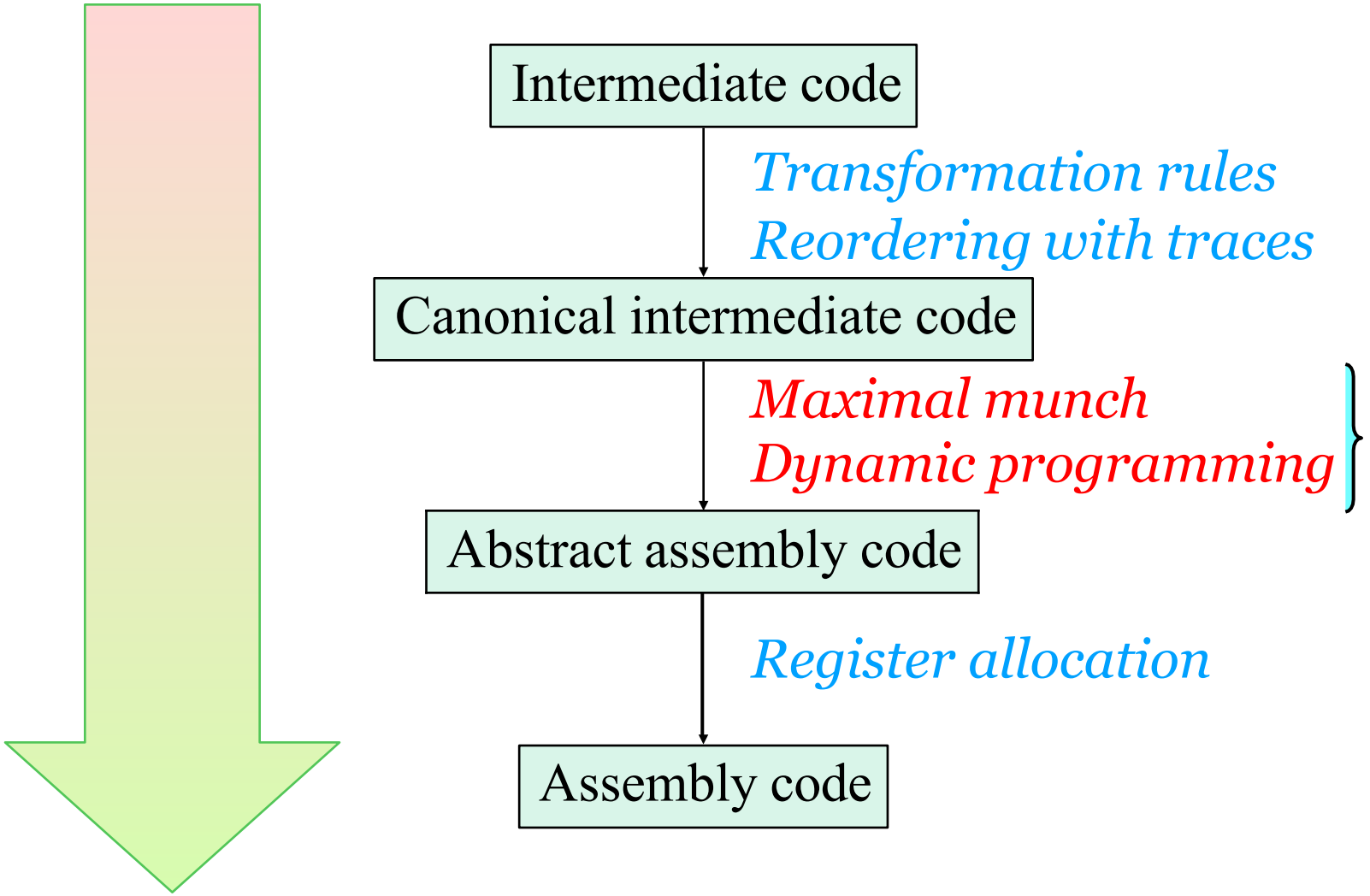
编译原理
9. 指令选择

rainoftime.github.io
浙江大学
计算机科学与技术学院

Content

Where We Are


Intermediate code
Transformation rules
Reordering with traces
Canonical intermediate code
Maximal munch
Dynamic programming
Abstract assembly code
Register allocation
Assembly code

Outline

1. Overview of Instruction Selection
2. Algorithms for Instruction Selection
3. CISC vs RISC

1. Overview of Instruction Selection

- Instruction Selection
- Instruction Sel. via Tree Covering
- Optimal and Optimum Tilings

Typical Tasks of "Compiler Backbend"

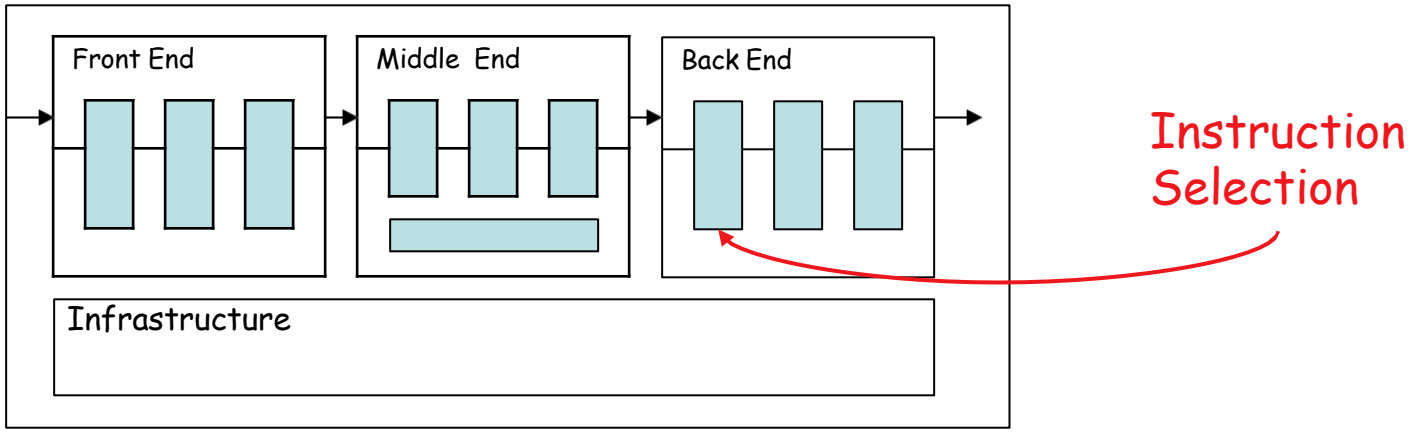
- **Instruction Selection**
  Mapping *IR* into abstract assembly code

- **Register Allocation**
  Deciding which values will reside in registers

- **Instruction Scheduling (Not covered)**
  Reordering operations to hide latencies, exploit parallelism, etc.

Instruction Selection: The Problem

- **Mapping *IR* into abstract assembly code**
- Abstract assembly = assembly with infinite registers
  - Invent new temporaries for intermediate results
  - Map to actual registers later



Instruction Selection: The Problem

1. **How do we represent the capabilities of machine instructions?**

2. **How do we cover an IR tree with available instructions?**

3. **How do we find the best (least cost) covering?**

The Essence of Instruction Selection

Performing some kind of "pattern matching"

- Macro Expansion
- Tree Covering
  - Maximal munch
  - Dynamic programming
  - Tree grammar
- DAG Covering
- …?

Tree-oriented IR suggests pattern matching on tree

1. Overview of Instruction Selection

- Instruction Selection
- Instruction Sel. via Tree Covering
- Optimal and Optimum Tilings

The Jouette Instruction Set

- For illustration, we use the Jouette architecture

Name	*Effect*	
—	r_i	
ADD	r_i	$r_j + r_k$
MUL	r_i	$r_j \times r_k$
SUB	r_i	$r_j - r_k$
DIV	r_i	r_j / r_k
ADDI	r_i	$r_j + c$
SUBI	r_i	$r_j - c$
LOAD	r_i	$M[r_j + c]$

Name	*Effect*	
—	M[]	
STORE	$M[r_j + c]$	r_i
MOVEM	$M[r_j]$	$M[r_i]$

The Jouette Instruction Set

- RISC-style, load/store architecture
- Relative large, general purpose register file
  - Data or address can reside in registers
  - Each instruction can access any register
- Register r_0 always contains zero
- Each instruction has latency of **one cycle**
  - Except MOVEM (which is m)
- Execution of only one instruction per cycle

Instruction Selection via Tree Patterns

- **Tree pattern**
  - A fragment of an IR tree that corresponds to a single machine instruction.
  - A tree pattern is also called a tile.
- **Instruction selection = Tiling**
  - Cover the entire IR tree with non-overlapping tiles, where each tile maps to one legal machine instruction.

将IR与后端的机器指令**都转换为树结构**。这样就把指令选择问题转换为机器指令树覆盖全IR Tree的问题。

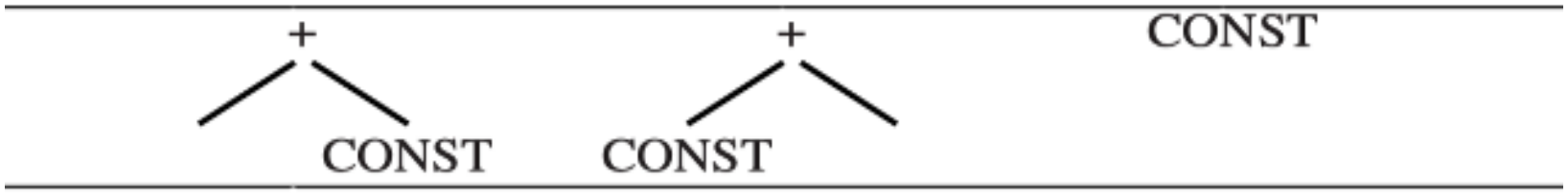
Tree Patterns: Jouette

Name	*Effect*	*Trees*
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	+
MUL	$r_i \leftarrow r_j \times r_k$	*
SUB	$r_i \leftarrow r_j - r_k$	-
DIV	$r_i \leftarrow r_j / r_k$	/
ADDI	$r_i \leftarrow r_j + c$	+ (CONST) + (CONST) CONST
SUBI	$r_i \leftarrow r_j - c$	- (CONST)
LOAD	$r_i \leftarrow M[r_j + c]$	MEM + (CONST) MEM + (CONST) MEM (CONST) MEM
STORE	$M[r_j + c] \leftarrow r_i$	MOVE (MEM + (CONST)) MOVE (MEM + (CONST)) MOVE (MEM (CONST)) MOVE (MEM)
MOVEM	$M[r_j] \leftarrow M[r_i]$	MOVE (MEM) (MEM)

- Some instructions correspond to more than one tree pattern
- The actual values of CONST and TEMP nodes are not always be shown.

Multiple Patterns: ADDI Example

- Some instructions have **multiple** tree patterns due to commutativity of + and ×, and the special case $r_0 = 0$.
- E.g., the ADDI $r_i \leftarrow r_j + c$ instruction matches **three** tree patterns:



- The third pattern: ADDI $r_i \leftarrow r_0 + c$, since $r_0 = 0$.

The TEMP "Tile" — No Instruction Needed

- A TEMP node represents a value already in a register
- It "produces a result in a register" without executing any instruction
- This is the **first entry** in the Jouette table (the "—" row)

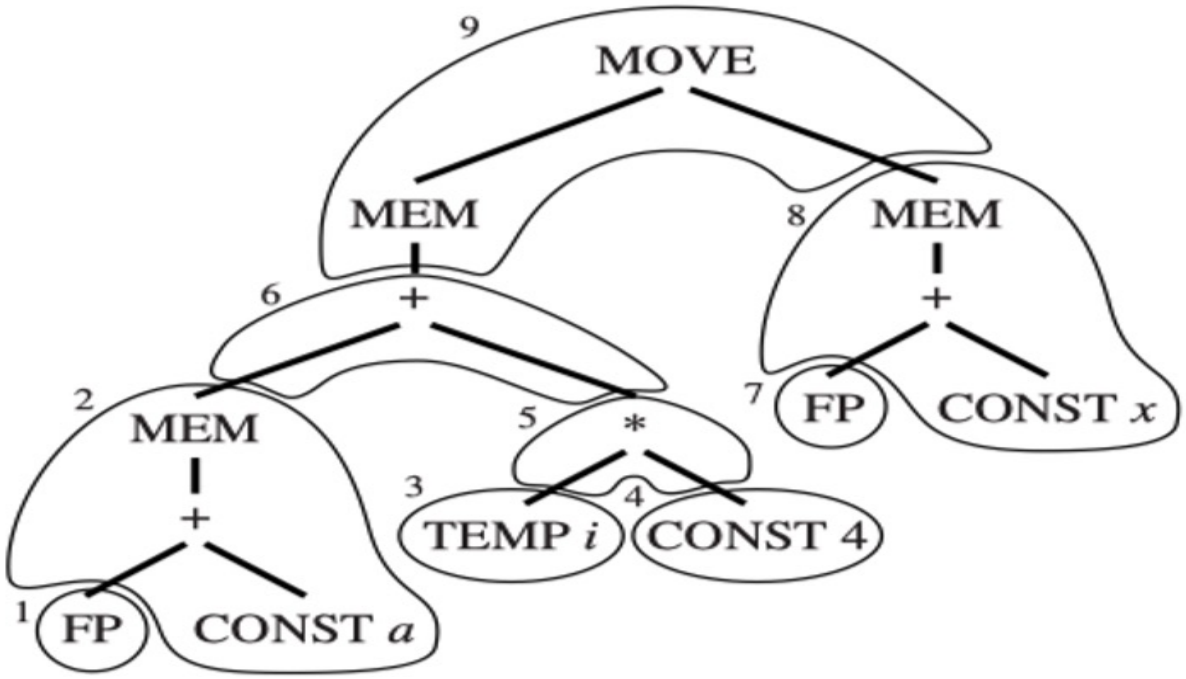
Important: TEMP tiles are "free" — they cost zero instructions. They simply reference an existing register.

Instruction Selection as Tiling

- Each tile covers one or more tree nodes
- Tiles must **not overlap**
- Together they must cover **every** node
- Each tile → one machine instruction (except TEMP tiles → zero instructions)

Instruction Selection as Tiling

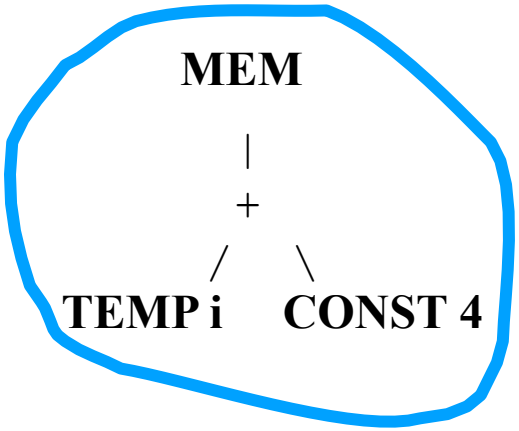
- **Tiling**: cover the IR tree with a set of non-overlapping tree patterns



可以想象为铺地板的过程，每一个tree pattern就是各种大小不一的瓷砖（tile），指令选择就类似用瓷砖铺满屋子

Example: Tiling for a Memory Load

- **Tiling**: cover the IR tree with a set of non-overlapping tree patterns
  - E.g., the memory load can be matched with different patterns

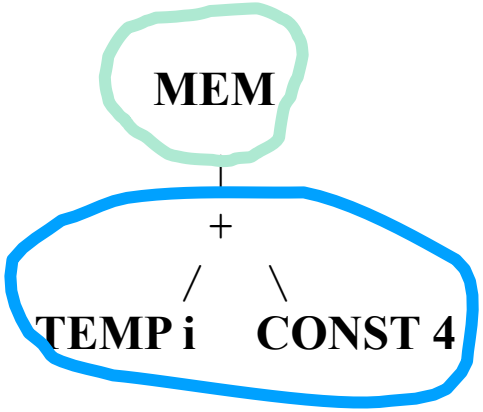


ADDI	$r_i \leftarrow r_j + c$	+ CONST, + CONST, CONST
SUBI	$r_i \leftarrow r_j - c$	- CONST
LOAD	$r_i \leftarrow M[r_j + c]$	MEM + CONST, MEM + CONST, MEM CONST, MEM

The IR Tree expresses only one operation in each tree node (e.g., fetch), but a real machine instruction can perform **several of primitive operations.**

Example: Tiling for a Memory Load

- **Tiling**: cover the IR tree with a set of non-overlapping tree patterns
  - E.g., the memory load can be matched with different patterns

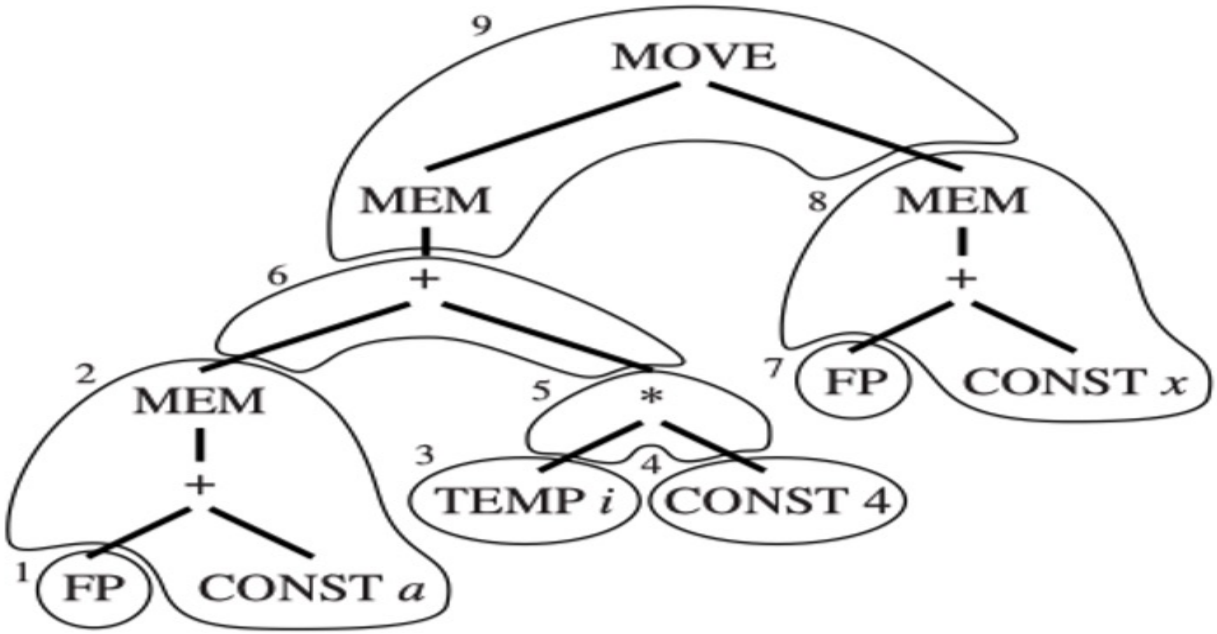


ADDI	$r_i \leftarrow r_j + c$	+ CONST, + CONST, CONST
SUBI	$r_i \leftarrow r_j - c$	- CONST
LOAD	$r_i \leftarrow M[r_j + c]$	MEM + CONST, MEM + CONST, MEM CONST, MEM

Example: Tiling (The Example at the Beginning)

- **a[i] := x**, assuming i in register, a and x in stack frame
  - The a is the frame offset of the pointer to the array.

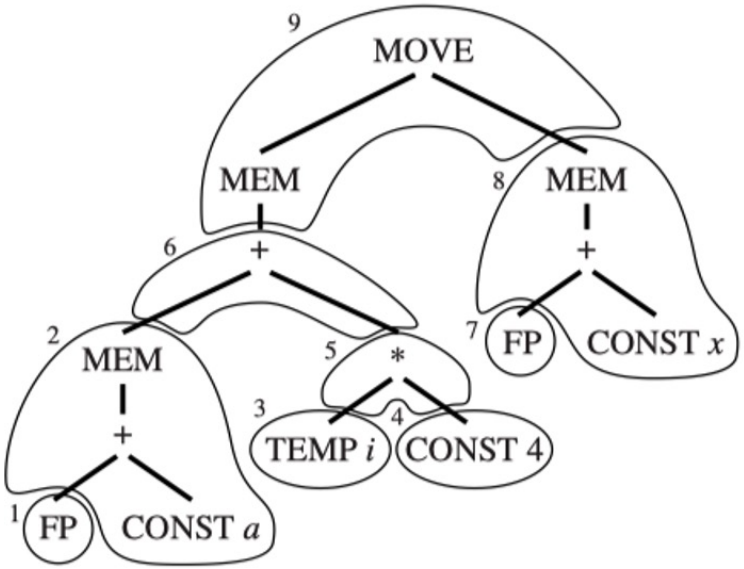
*M[a+i*4]:=x:* each element takes up 4 storage locations



IR Tree and one possible tiling option for it

Example: Instruction Emission

- **a[i] := x**, assuming i in register, a and x in stack frame



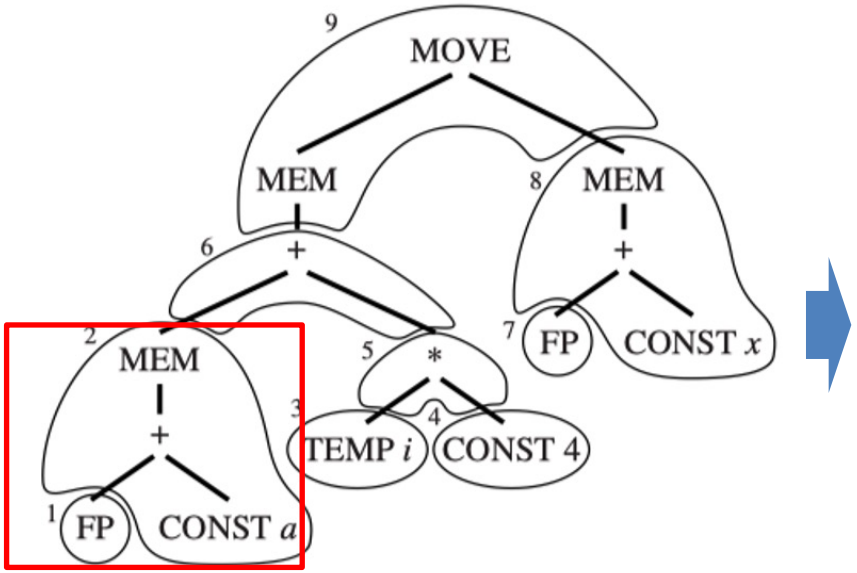
2	LOAD	r1 ← M[fp + a]
4	ADDI	r2 ← r0 + 4
5	MUL	r2 ← ri × r2
6	ADD	r1 ← r1 + r2
8	LOAD	r2 ← M[fp + x]
9	STORE M	[r1 + 0] ← r2

Instructions (6 total)

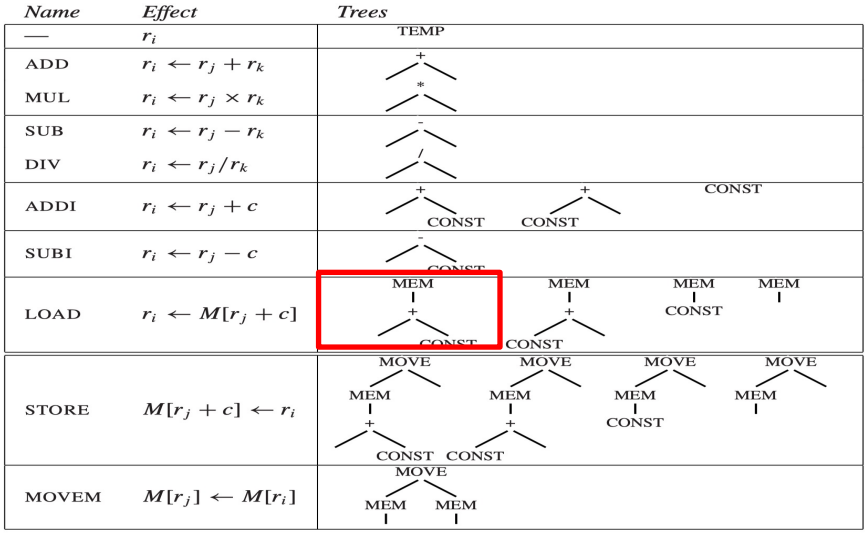
NOTE: Tiles 1, 3, and 7 do not correspond to any machine instructions, because they are just (virtual) registers (TEMPs).

Example: Instruction Emission

- **a[i] := x**, assuming i in register, a and x in stack frame



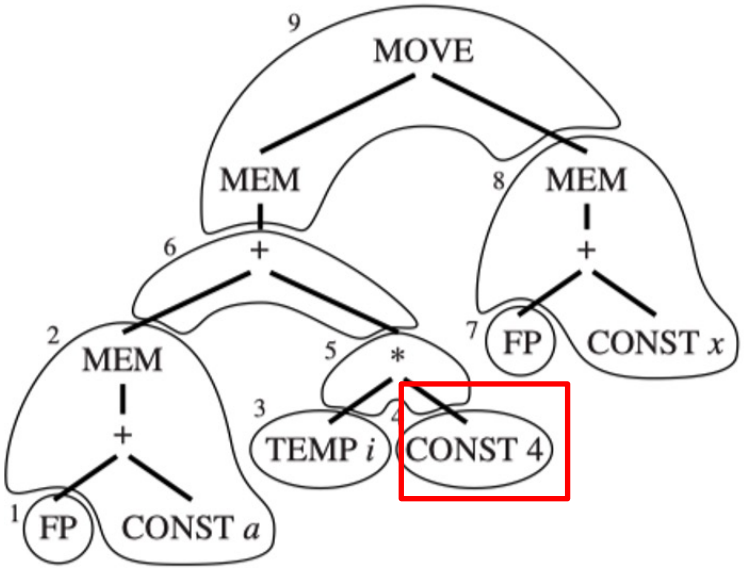
2	LOAD	r1 ← M[fp + a]
4	ADDI	r2 ← r0 + 4
5	MUL	r2 ← ri × r2
6	ADD	r1 ← r1 + r2
8	LOAD	r2 ← M[fp + x]
9	STORE M	[r1 + 0] ← r2



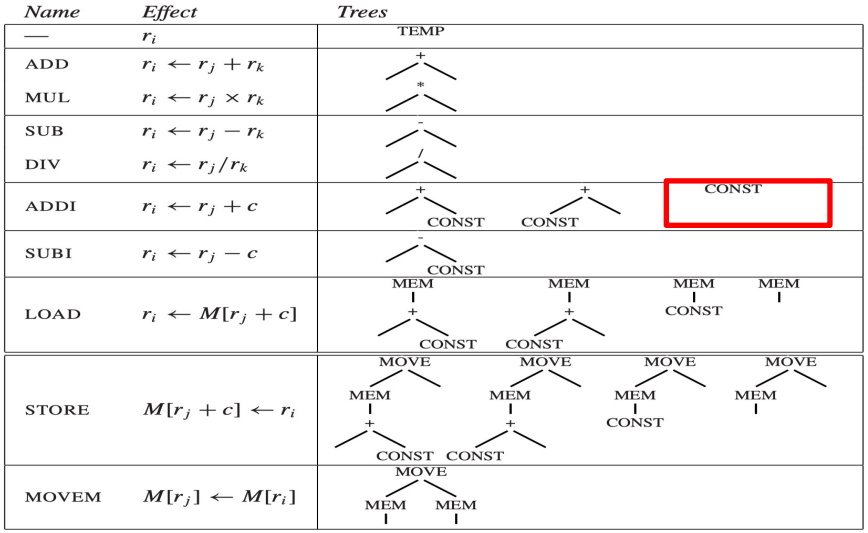
Tiles connected by new temporary registers that hold result of tile

Example: Instruction Emission

- **a[i] := x**, assuming i in register, a and x in stack frame



2	LOAD	r1 ← M[fp + a]
4	ADDI	r2 ← r0 + 4
5	MUL	r2 ← ri × r2
6	ADD	r1 ← r1 + r2
8	LOAD	r2 ← M[fp + x]
9	STORE M	[r1 + 0] ← r2

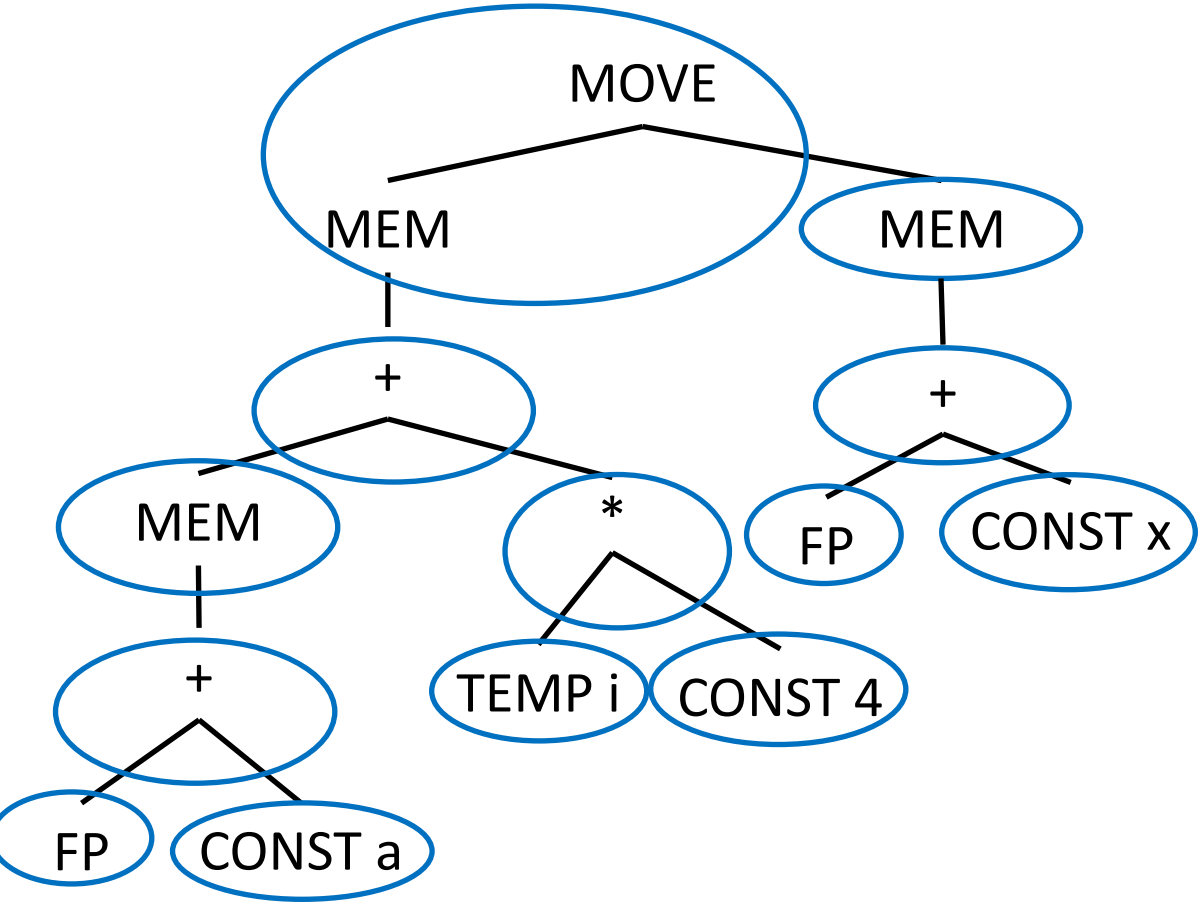


可以认为CONST 4需要一个额外的指令来计算，也就是**ADDI r2 ← r0 + 4**

We will discuss instruction emission later!

Example: Tiling with Another Strategy

- **a[i] := x** is always possible to tile the tree with tiny tiles, each covering only one node.




```
ADDI    r1 ← r0 + a
ADD     r1 ← fp + r1
LOAD    r1 ← M[r1 + 0]
ADDI    r2 ← r0 + 4
MUL     r2 ← ri × r2
ADD     r1 ← r1 + r2
ADDI    r2 ← r0 + x
ADD     r2 ← fp + r2
LOAD    r2 ← M[r2 + 0]
STORE   M[r1 + 0] ← r2
```

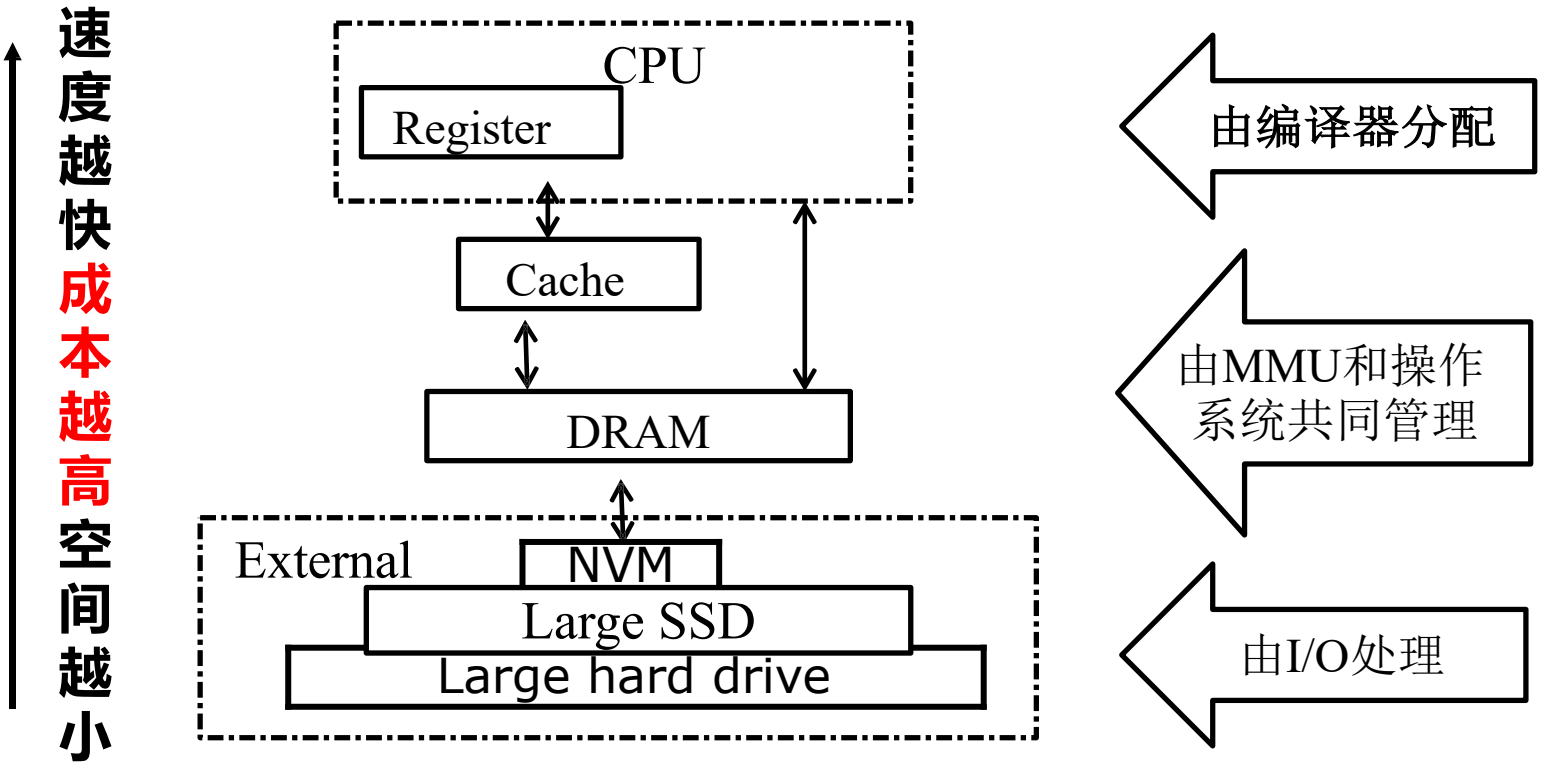
只要我们的tree pattern足够小且丰富（比如能够覆盖单一节点），那么就很有机会把程序对应的IR tree覆盖完整

1. Overview of Instruction Selection

- Instruction Selection
- Instruction Sel. via Tree Covering
- Optimal and Optimum Tilings

Instruction Selection Criteria

- **Need a good selection of tiles**
  - Small tiles to make sure we can tile every tree
  - Pick large tiles => potentially fewer instruction, but instructions ≠ cycles



理想情况需考虑指令代价、运算对象和结果如何存储等多重因素

Optimal and Optimum Tilings

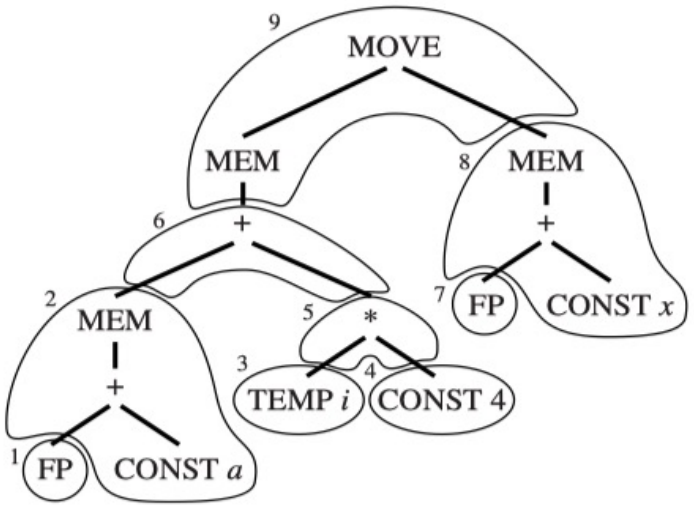
- **Optimum tiling**
  - Tiles sum to the lowest possible value.
  - Globally "the best" (global minimum)

- **Optimal tiling**
  - No wo adjacent tiles can be combined into a single tile of lower cost
  - Locally "the best" (local minimum)

Every optimum tiling is also optimal, but not vice versa

Example: Optimal and Optimum Tilings

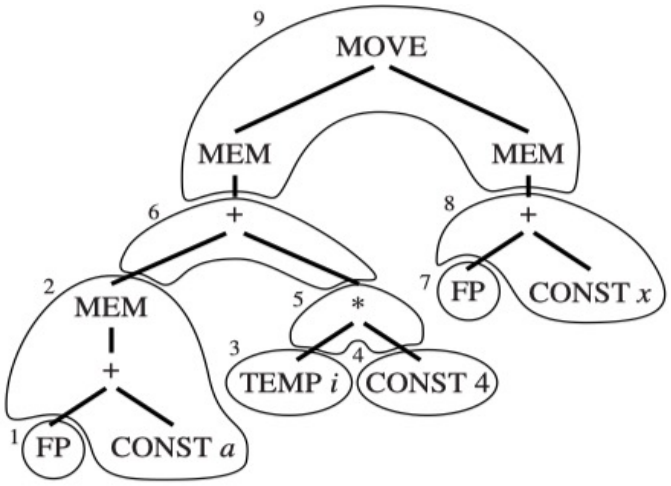
- Suppose every instruction costs one unit, except for MOVEM which costs m units.



2	LOAD	$r_1 \leftarrow M[\mathbf{fp} + a]$
4	ADDI	$r_2 \leftarrow r_0 + 4$
5	MUL	$r_2 \leftarrow r_i \times r_2$
6	ADD	$r_1 \leftarrow r_1 + r_2$
8	LOAD	$r_2 \leftarrow M[\mathbf{fp} + x]$
9	STORE	$M[r_1 + 0] \leftarrow r_2$

(a)

6 units



2	LOAD	$r_1 \leftarrow M[\mathbf{fp} + a]$
4	ADDI	$r_2 \leftarrow r_0 + 4$
5	MUL	$r_2 \leftarrow r_i \times r_2$
6	ADD	$r_1 \leftarrow r_1 + r_2$
8	ADDI	$r_2 \leftarrow \mathbf{fp} + x$
9	MOVEM	$M[r_1] \leftarrow M[r_2]$

(b)

5+m units

Discussion: Instruction Cost and Optimality

- In reality, instructions are not self-contained with individually attributable costs,
- The term “Optimum tiling” is based on an idealized cost model!
- Real cost factors include:
  - Pipeline interactions
  - Register allocation effects
  - Cache behavior
  - Instruction scheduling opportunities
  - …

2. Algorithms for Instruction Selection

- Maximal Munch
- Dynamic Programming
- Tree Grammar

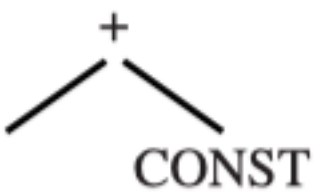
Algorithms for Instruction Selection

- **Maximal Munch**: Find an optimal tiling
  - Greedy, top-down strategy
  - Cover the current node with the largest tile, and repeat on subtrees
- **Dynamic Programming**: Find an optimum tiling
  - Bottom-up strategy
  - Assign cost to each node
  - Cost = cost of selected tile + cost of subtrees
  - Select a tile with minimal cost and recurse upward

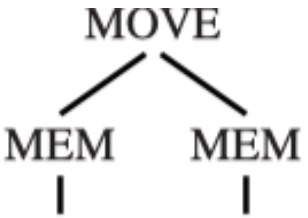
Maximal Munch: Greedy Tiling

- Basic assumption: **larger tiles = better**
- **Largest tile**: the one with the most nodes
  - Tiles of equivalent size => arbitrarily choose one

one node



two nodes



three nodes

- **Main idea:** Start from top of tree and use largest tile that matches tree. Tile remaining subtrees recursively

Maximal Munch: The Algorithm

A top-down traversal for greedy tiling

1. Start at the root of the tree
2. Find the largest tile pattern that matches at this node
   - If two tiles of equal size match, choice may be arbitrary or cost-based
3. Cover the root (and nearby nodes) with this tile
4. Several uncovered subtrees remain
5. Repeat for each subtree

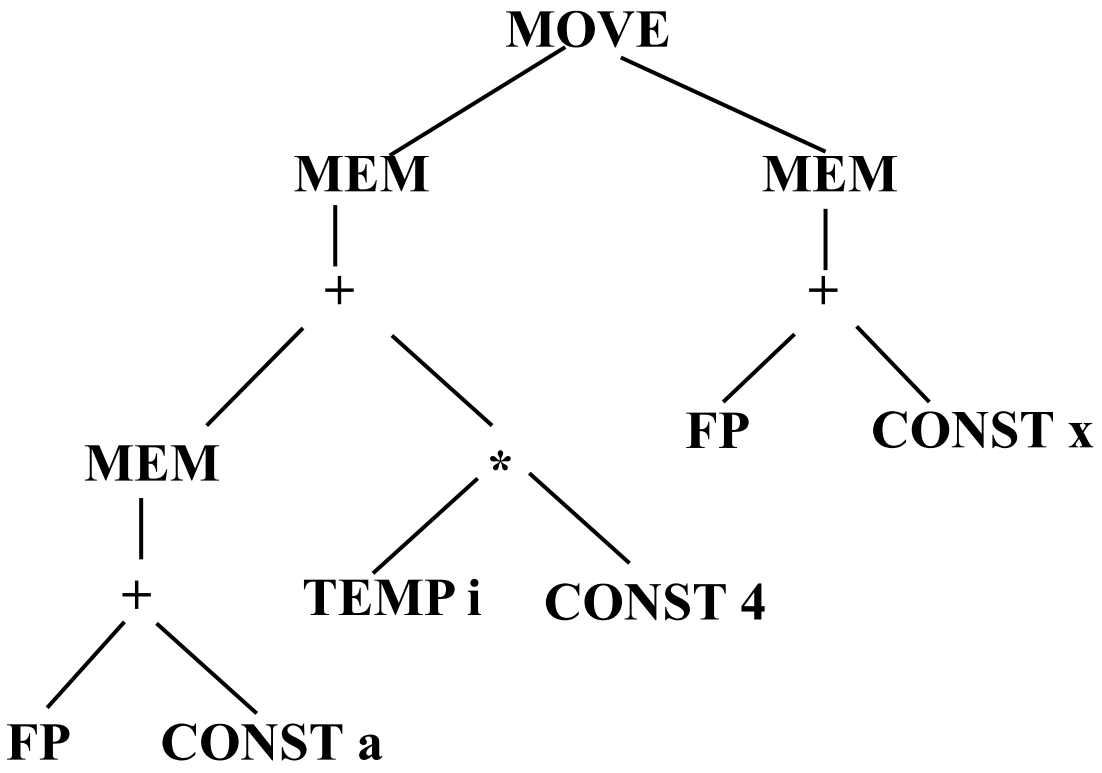
Strategy: At each node, always pick the *largest* tile that fits. Then recurse on the remaining subtrees.

Example: Maximal Munch

- **a[i] := x**, assuming i in register, a and x in stack frame

*M[a+i*4]:=x*

Name	*Effect*	*Trees*
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	+
MUL	$r_i \leftarrow r_j \times r_k$	*
SUB	$r_i \leftarrow r_j - r_k$	-
DIV	$r_i \leftarrow r_j / r_k$	/
ADDI	$r_i \leftarrow r_j + c$	+ CONST, + CONST, CONST
SUBI	$r_i \leftarrow r_j - c$	- CONST
LOAD	$r_i \leftarrow M[r_j + c]$	MEM + CONST, MEM + CONST, MEM CONST, MEM
STORE	$M[r_j + c] \leftarrow r_i$	MOVE MEM + CONST CONST, MOVE MEM + CONST CONST, MOVE MEM CONST, MOVE MEM
MOVEM	$M[r_j] \leftarrow M[r_i]$	MOVE MEM MEM



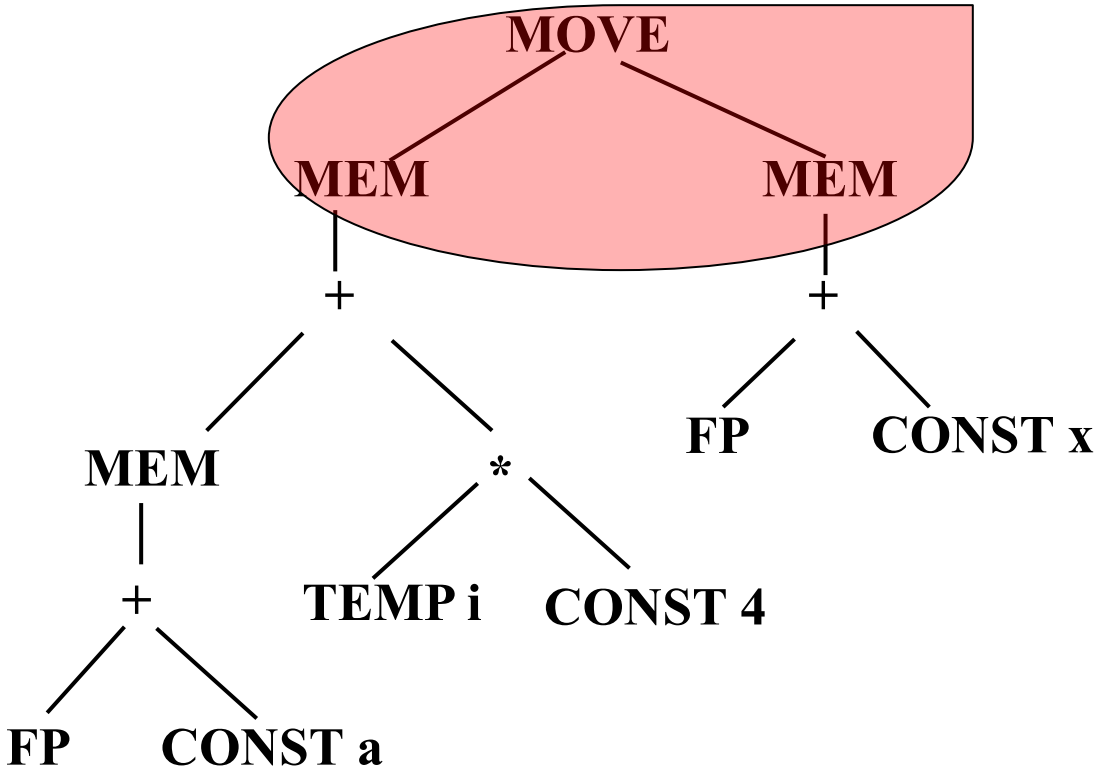
Example: Maximal Munch

Name	*Effect*	*Trees*
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	+
MUL	$r_i \leftarrow r_j \times r_k$	*
SUB	$r_i \leftarrow r_j - r_k$	-
DIV	$r_i \leftarrow r_j / r_k$	/
ADDI	$r_i \leftarrow r_j + c$	+ CONST, + CONST, CONST
SUBI	$r_i \leftarrow r_j - c$	- CONST
LOAD	$r_i \leftarrow M[r_j + c]$	MEM + CONST, MEM + CONST, MEM CONST, MEM
STORE	$M[r_j + c] \leftarrow r_i$	MOVE MEM + CONST CONST, MOVE MEM + CONST CONST, MOVE MEM CONST, MOVE MEM
MOVEM	$M[r_j] \leftarrow M[r_i]$	MOVE MEM MEM

1. Start at the root node MOVE
   - Check for MOVEM pattern:

MOVE(MEM(e1), MEM(e2))

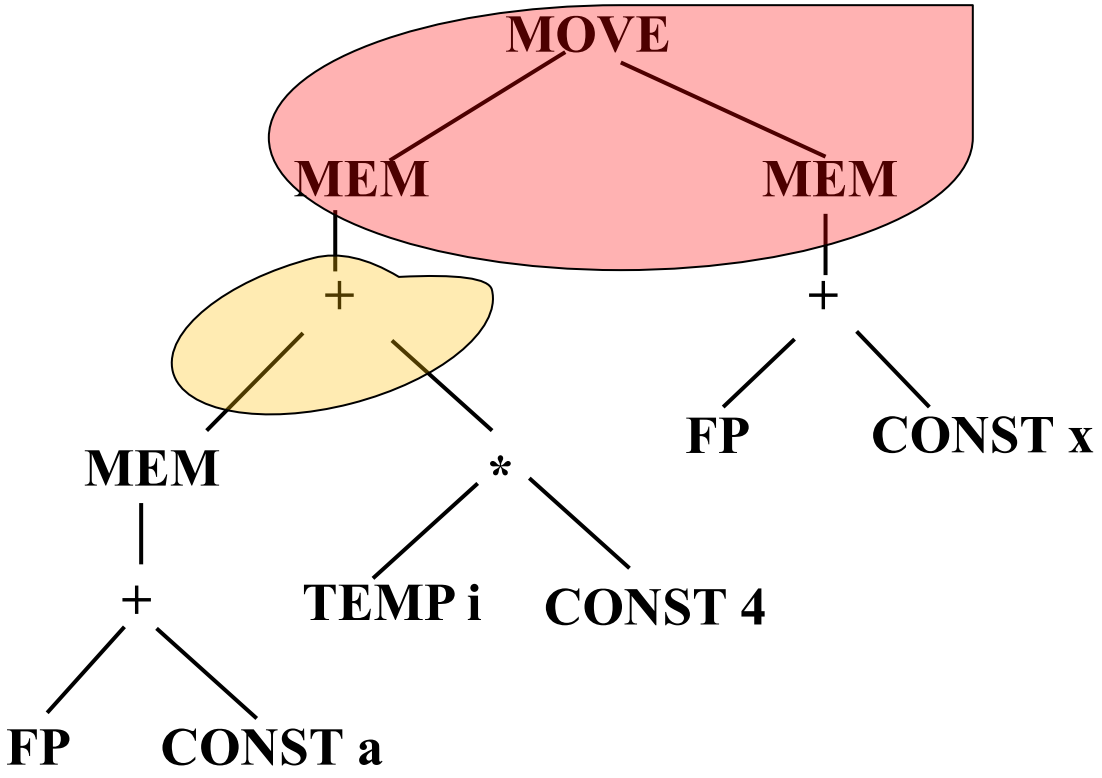
- **Covers 3 nodes!**



Example: Maximal Munch

Name	Effect	Trees
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	+
MUL	$r_i \leftarrow r_j \times r_k$	*
SUB	$r_i \leftarrow r_j - r_k$	-
DIV	$r_i \leftarrow r_j / r_k$	/
ADDI	$r_i \leftarrow r_j + c$	+ CONST, + CONST, CONST
SUBI	$r_i \leftarrow r_j - c$	- CONST
LOAD	$r_i \leftarrow M[r_j + c]$	MEM + CONST, MEM + CONST, MEM CONST, MEM
STORE	$M[r_j + c] \leftarrow r_i$	MOVE MEM + CONST CONST, MOVE MEM + CONST CONST, MOVE MEM CONST, MOVE MEM
MOVEM	$M[r_j] \leftarrow M[r_i]$	MOVE MEM MEM

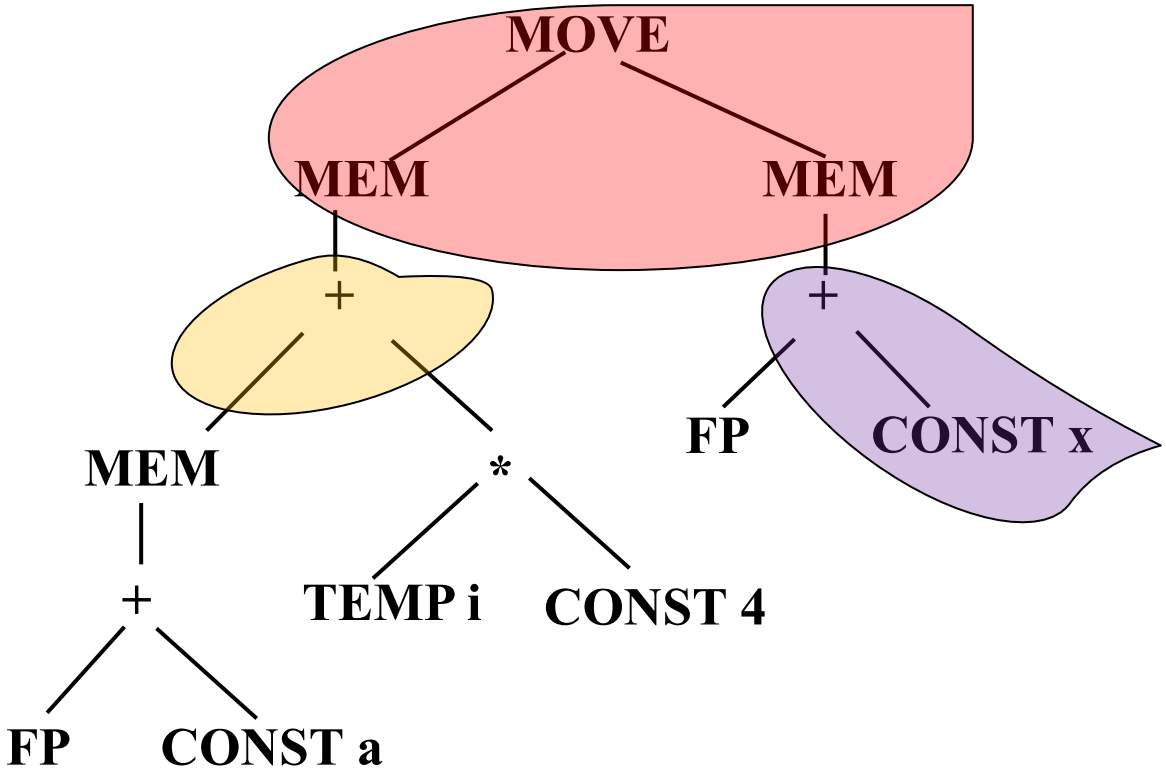
2. Process the left + node
   - Match the ADD pattern



Example: Maximal Munch

Name	*Effect*	*Trees*
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	+
MUL	$r_i \leftarrow r_j \times r_k$	*
SUB	$r_i \leftarrow r_j - r_k$	-
DIV	$r_i \leftarrow r_j / r_k$	/
ADDI	$r_i \leftarrow r_j + c$	+ CONST, + CONST, CONST
SUBI	$r_i \leftarrow r_j - c$	- CONST
LOAD	$r_i \leftarrow M[r_j + c]$	MEM + CONST, MEM + CONST, MEM CONST, MEM
STORE	$M[r_j + c] \leftarrow r_i$	MOVE MEM + CONST CONST, MOVE MEM + CONST CONST, MOVE MEM CONST, MOVE MEM
MOVEM	$M[r_j] \leftarrow M[r_i]$	MOVE MEM MEM

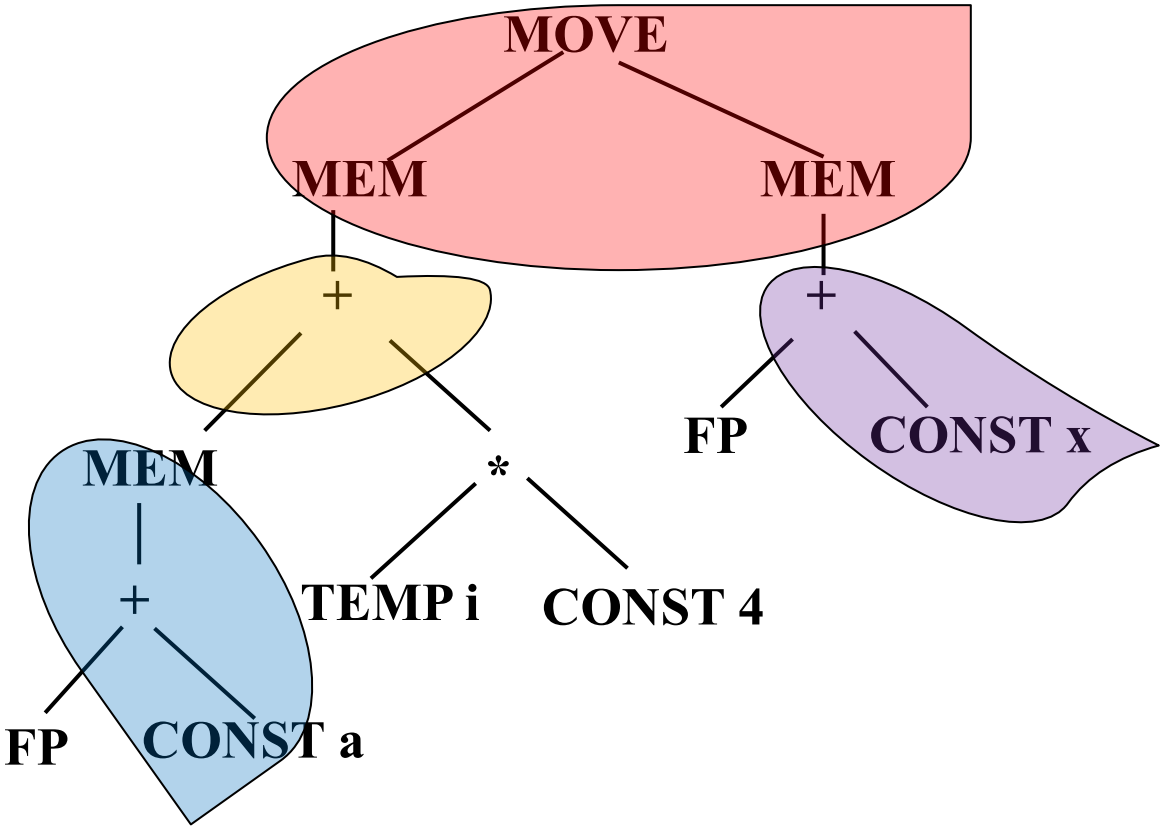
3. Process the right + node
   - Match the ADDI pattern



Example: Maximal Munch

Name	Effect	Trees
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	+
MUL	$r_i \leftarrow r_j \times r_k$	*
SUB	$r_i \leftarrow r_j - r_k$	-
DIV	$r_i \leftarrow r_j / r_k$	/
ADDI	$r_i \leftarrow r_j + c$	+ CONST; + CONST; CONST
SUBI	$r_i \leftarrow r_j - c$	- CONST
LOAD	$r_i \leftarrow M[r_j + c]$	MEM + CONST; MEM + CONST; MEM CONST; MEM
STORE	$M[r_j + c] \leftarrow r_i$	MOVE MEM + CONST CONST; MOVE MEM + CONST; MOVE MEM CONST; MOVE MEM
MOVEM	$M[r_j] \leftarrow M[r_i]$	MOVE MEM MEM

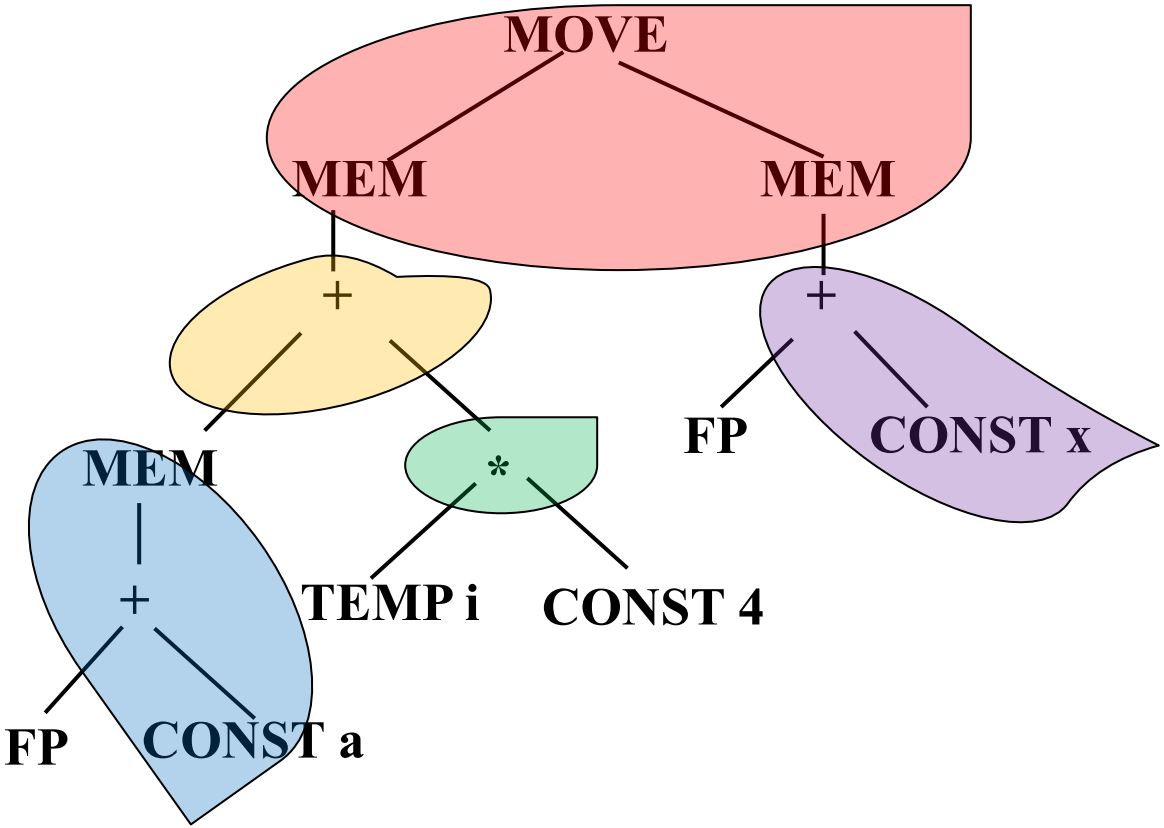
4. Process the left MEM node
   - Match the LOAD pattern



Example: Maximal Munch

Name	Effect	Trees
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	+
MUL	$r_i \leftarrow r_j \times r_k$	*
SUB	$r_i \leftarrow r_j - r_k$	-
DIV	$r_i \leftarrow r_j / r_k$	/
ADDI	$r_i \leftarrow r_j + c$	+ CONST, + CONST, CONST
SUBI	$r_i \leftarrow r_j - c$	- CONST
LOAD	$r_i \leftarrow M[r_j + c]$	MEM + CONST, MEM + CONST, MEM CONST, MEM
STORE	$M[r_j + c] \leftarrow r_i$	MOVE MEM + CONST CONST, MOVE MEM + CONST, MOVE MEM CONST, MOVE MEM
MOVEM	$M[r_j] \leftarrow M[r_i]$	MOVE MEM MEM

5. Process the * node
   - Match the MUL pattern

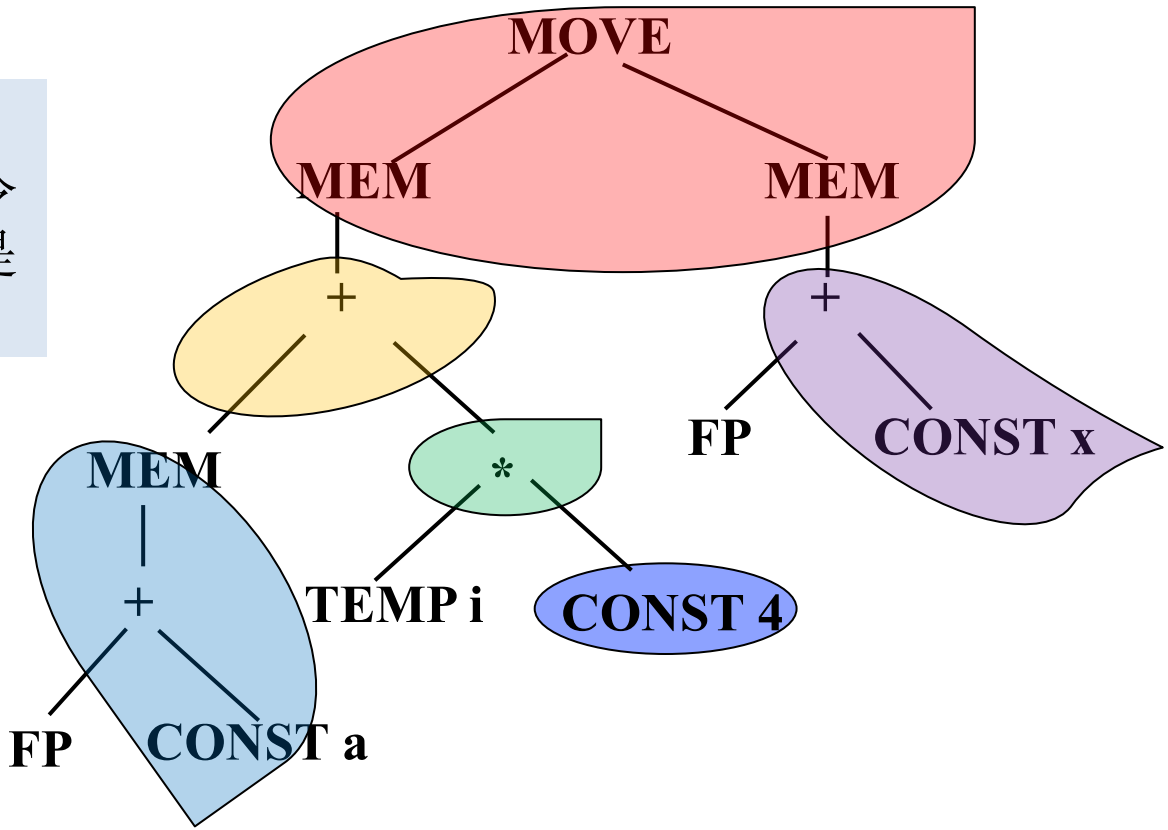


Example: Maximal Munch

Name	Effect	Trees
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	+
MUL	$r_i \leftarrow r_j \times r_k$	*
SUB	$r_i \leftarrow r_j - r_k$	-
DIV	$r_i \leftarrow r_j / r_k$	/
ADDI	$r_i \leftarrow r_j + c$	+ CONST; + CONST; CONST
SUBI	$r_i \leftarrow r_j - c$	- CONST
LOAD	$r_i \leftarrow M[r_j + c]$	MEM + CONST; MEM + CONST; MEM CONST; MEM
STORE	$M[r_j + c] \leftarrow r_i$	MOVE MEM + CONST CONST; MOVE MEM + CONST; MOVE MEM CONST; MOVE MEM
MOVEM	$M[r_j] \leftarrow M[r_i]$	MOVE MEM MEM

6. Process the CONST 4 node
   - Match the ADDI pattern

可以认为**CONST 4**需要一个额外的指令来计算，比如可能是
ADDI r2 ← r0 + 4



Maximal Munch – Instruction Emission

- Given a tiled tree, to generate code
  - Postorder traversal over tiles, not necessarily over individual IR nodes
  - For each tile, first emit code for its leaf subtrees
  - Then emit the instruction corresponding to the tile
  - Use the same temporary/register at tile boundaries

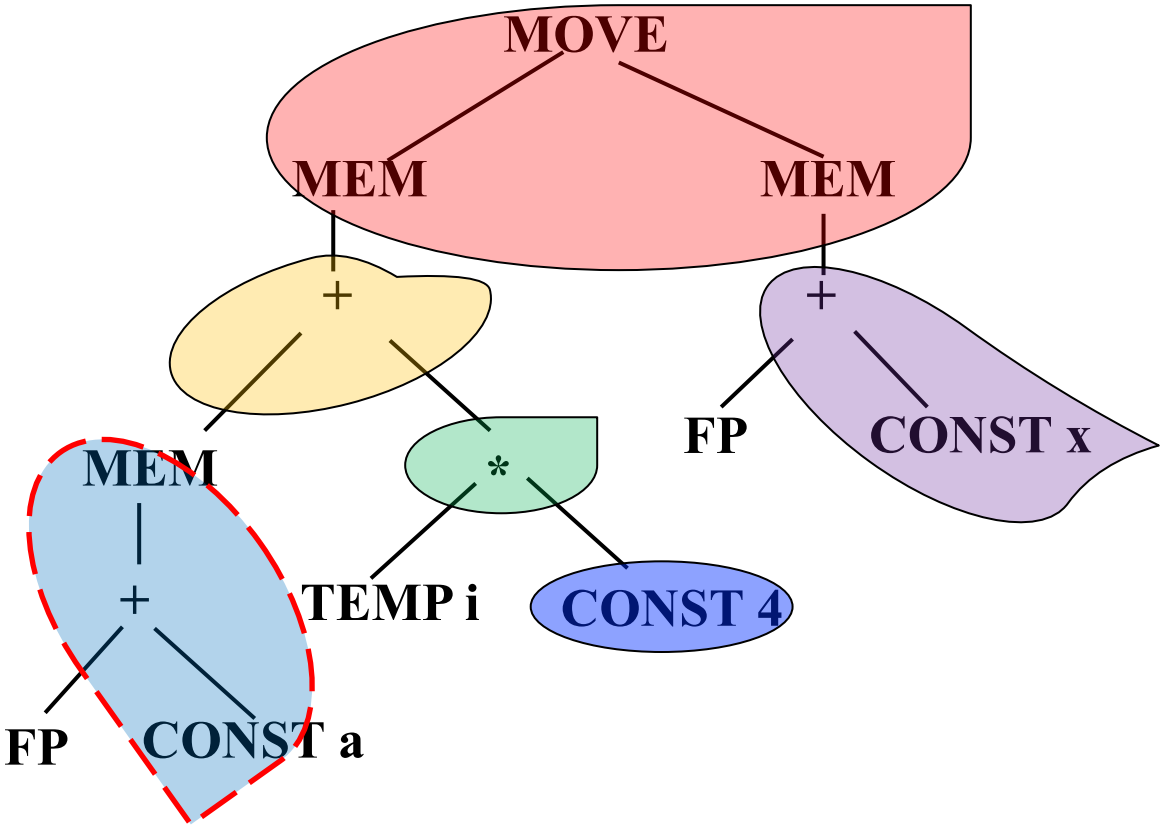
- Alternatively, interleave the tiling and codegen

Example: Instruction Emission

- **a[i] := x**, assuming i in register, a and x in stack frame (*M[a+i*4]:=x)*

Name	*Effect*	*Trees*
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	+
MUL	$r_i \leftarrow r_j \times r_k$	*
SUB	$r_i \leftarrow r_j - r_k$	-
DIV	$r_i \leftarrow r_j / r_k$	/
ADDI	$r_i \leftarrow r_j + c$	+ CONST; + CONST; CONST
SUBI	$r_i \leftarrow r_j - c$	- CONST
LOAD	$r_i \leftarrow M[r_j + c]$	MEM + CONST; MEM + CONST; MEM CONST; MEM
STORE	$M[r_j + c] \leftarrow r_i$	MOVE MEM + CONST CONST; MOVE MEM + CONST; MOVE MEM CONST; MOVE MEM
MOVEM	$M[r_j] \leftarrow M[r_i]$	MOVE MEM MEM

LOAD $r_1 \leftarrow M[fp+a]$



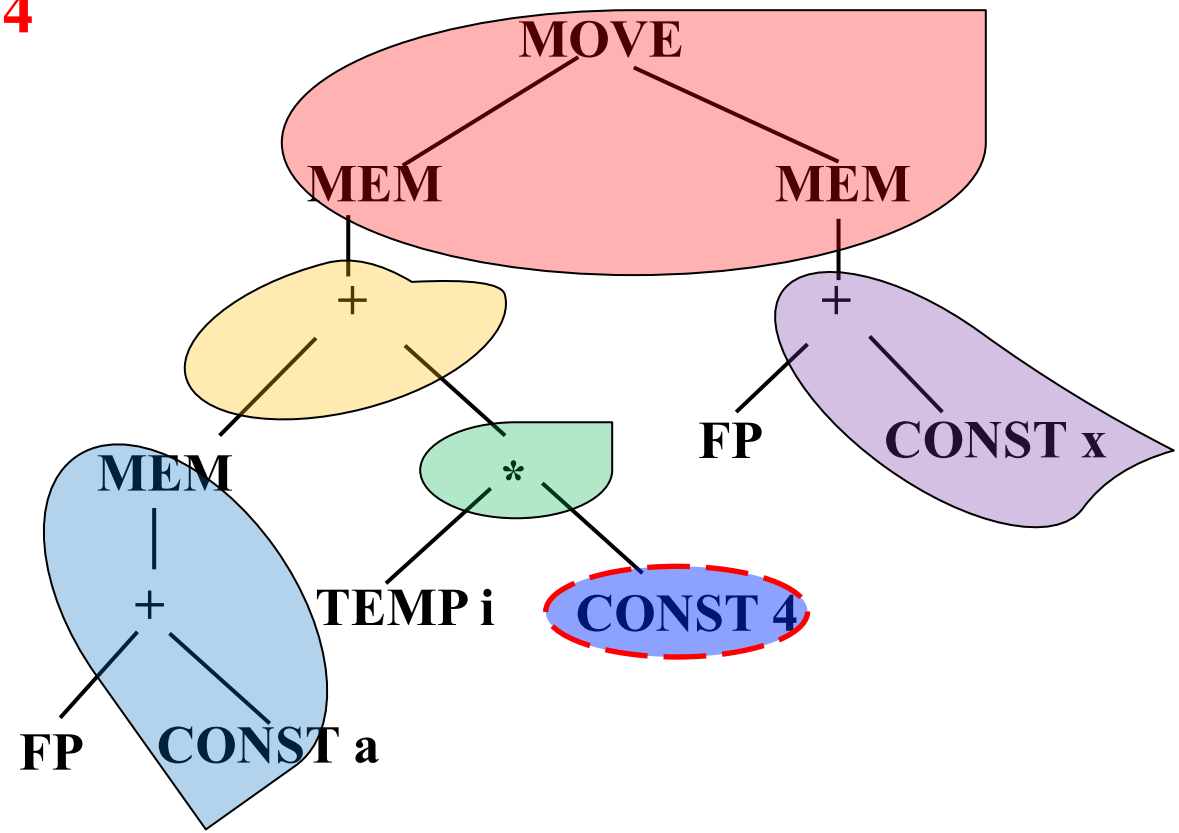
Example: Instruction Emission

- **a[i] := x**, assuming i in register, a and x in stack frame (*M[a+i*4]:=x)*

Name	*Effect*	*Trees*
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	+
MUL	$r_i \leftarrow r_j \times r_k$	*
SUB	$r_i \leftarrow r_j - r_k$	-
DIV	$r_i \leftarrow r_j / r_k$	/
ADDI	$r_i \leftarrow r_j + c$	+ CONST, + CONST, CONST
SUBI	$r_i \leftarrow r_j - c$	- CONST
LOAD	$r_i \leftarrow M[r_j + c]$	MEM + CONST, MEM + CONST, MEM CONST, MEM
STORE	$M[r_j + c] \leftarrow r_i$	MOVE MEM + CONST CONST, MOVE MEM + CONST, MOVE MEM CONST, MOVE MEM
MOVEM	$M[r_j] \leftarrow M[r_i]$	MOVE MEM MEM

LOAD $r_1 \leftarrow M[fp+a]$

ADDI $r_2 \leftarrow r_0 + 4$



Example: Instruction Emission

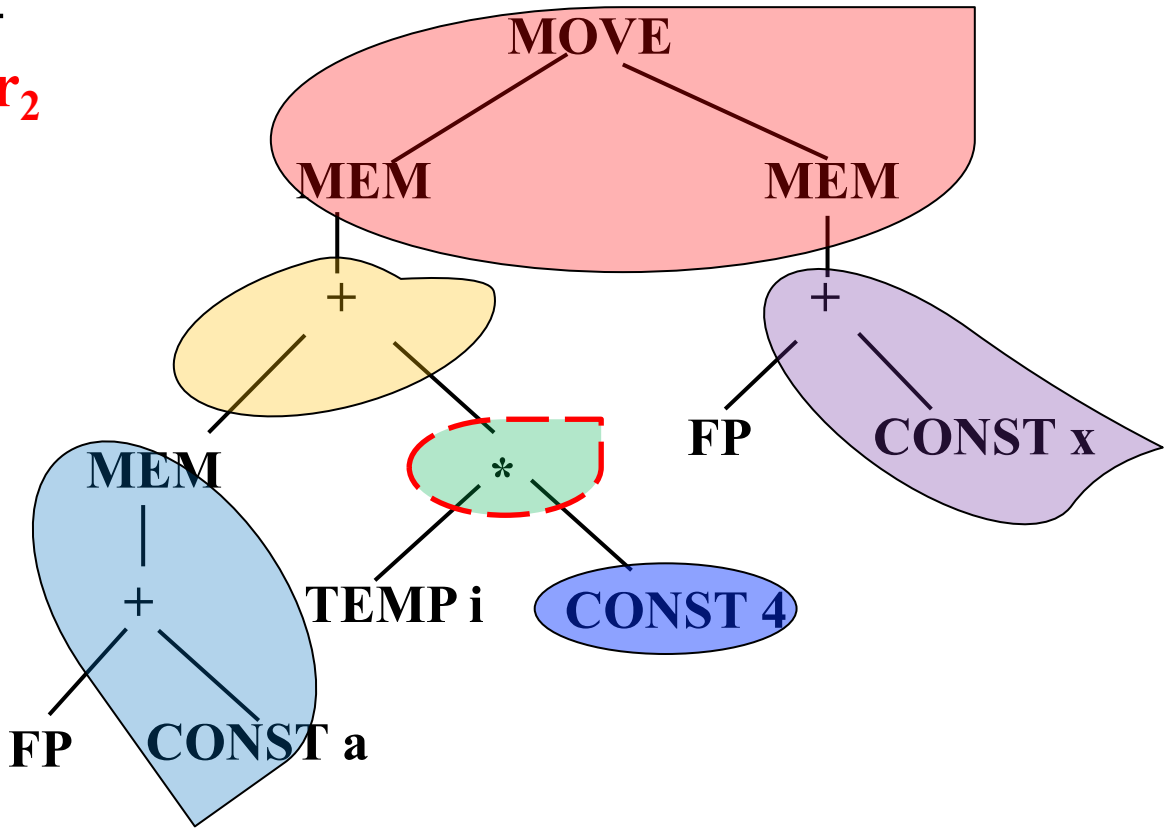
- **a[i] := x**, assuming i in register, a and x in stack frame (*M[a+i*4]:=x)*

Name	*Effect*	*Trees*
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	+
MUL	$r_i \leftarrow r_j \times r_k$	*
SUB	$r_i \leftarrow r_j - r_k$	-
DIV	$r_i \leftarrow r_j / r_k$	/
ADDI	$r_i \leftarrow r_j + c$	+ CONST, + CONST, CONST
SUBI	$r_i \leftarrow r_j - c$	- CONST
LOAD	$r_i \leftarrow M[r_j + c]$	MEM + CONST, MEM + CONST, MEM CONST, MEM
STORE	$M[r_j + c] \leftarrow r_i$	MOVE MEM + CONST CONST, MOVE MEM + CONST CONST, MOVE MEM CONST, MOVE MEM
MOVEM	$M[r_j] \leftarrow M[r_i]$	MOVE MEM MEM

LOAD $r_1 \leftarrow M[fp+a]$

ADDI $r_2 \leftarrow r_0 + 4$

MUL $r_2 \leftarrow r_i * r_2$

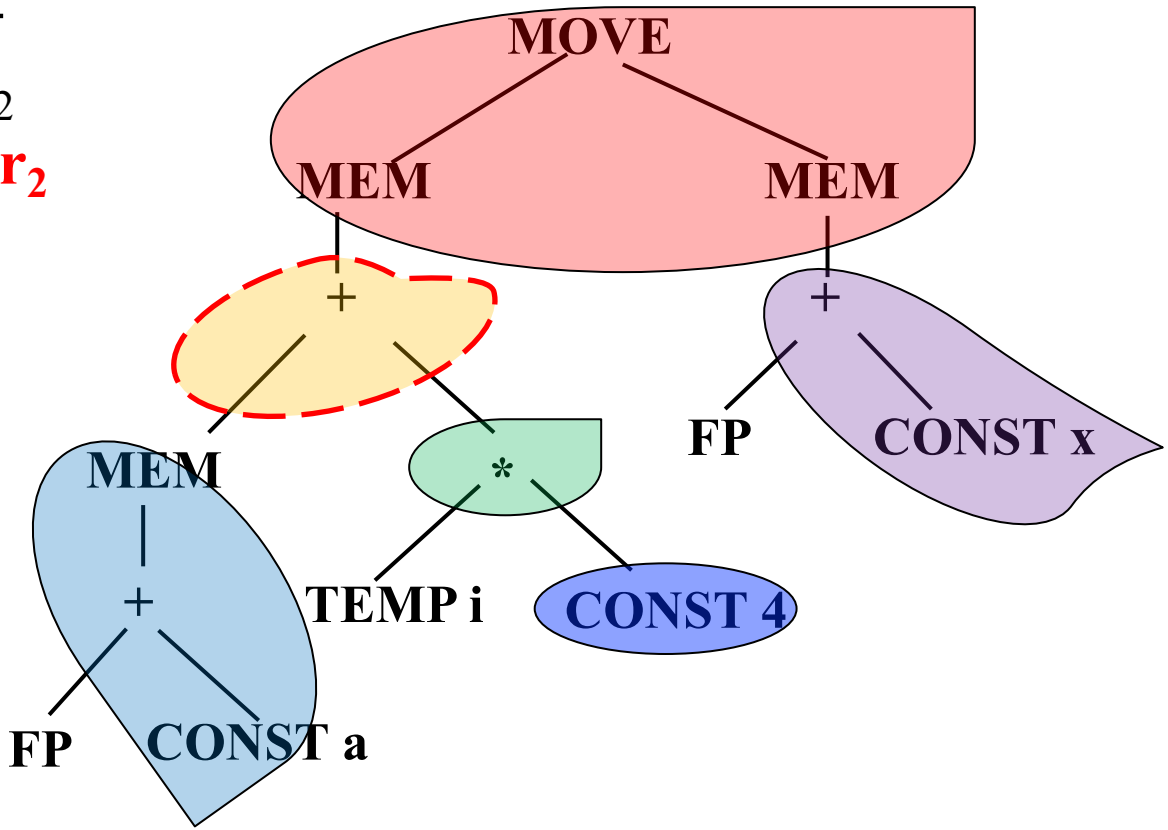


Example: Instruction Emission

- **a[i] := x**, assuming i in register, a and x in stack frame (*M[a+i*4]:=x)*

Name	*Effect*	*Trees*
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	+
MUL	$r_i \leftarrow r_j \times r_k$	*
SUB	$r_i \leftarrow r_j - r_k$	-
DIV	$r_i \leftarrow r_j / r_k$	/
ADDI	$r_i \leftarrow r_j + c$	+ CONST
		+ CONST
SUBI	$r_i \leftarrow r_j - c$	- CONST
LOAD	$r_i \leftarrow M[r_j + c]$	MEM + CONST
		MEM + CONST
		MEM CONST
		MEM
STORE	$M[r_j + c] \leftarrow r_i$	MOVE MEM + CONST CONST
		MOVE MEM + CONST
		MOVE MEM CONST
		MOVE MEM
MOVEM	$M[r_j] \leftarrow M[r_i]$	MOVE MEM MEM

LOAD $r_1 \leftarrow M[fp+a]$
ADDI $r_2 \leftarrow r_0 + 4$
MUL $r_2 \leftarrow r_i * r_2$
ADD $r_1 \leftarrow r_1 + r_2$

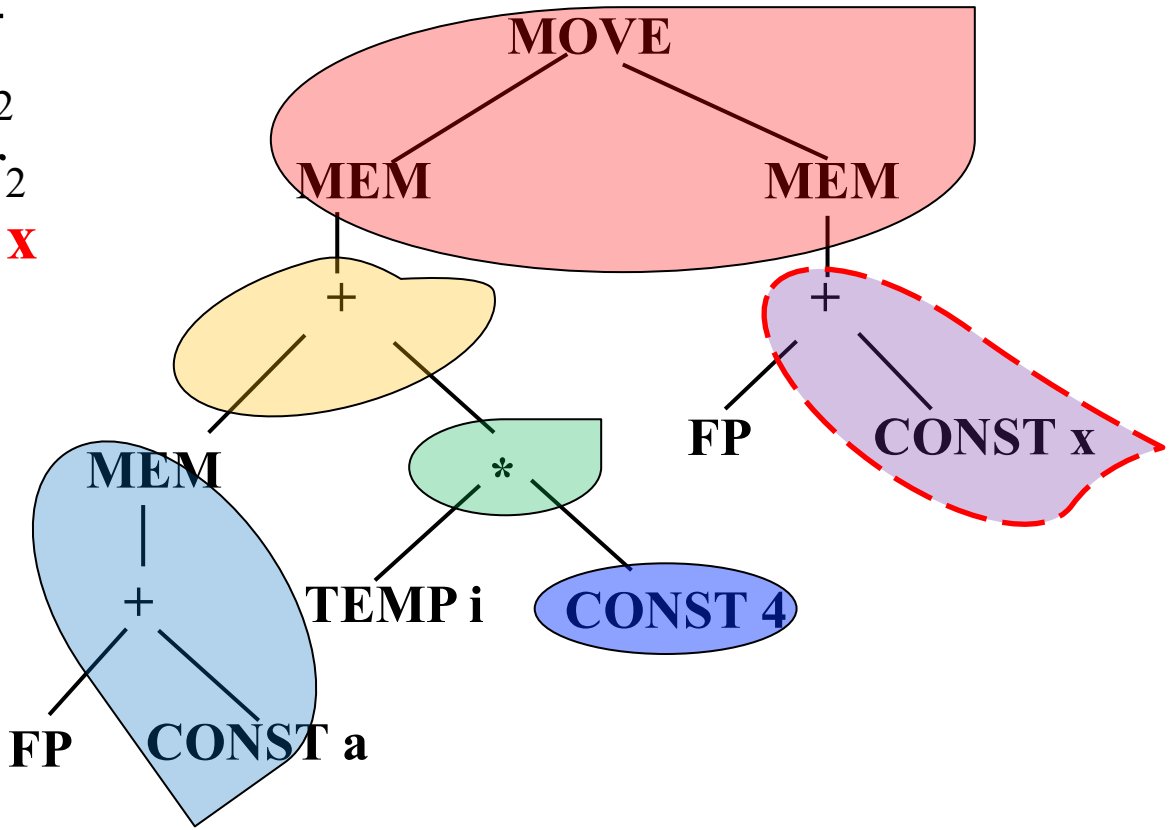


Example: Instruction Emission

- **a[i] := x**, assuming i in register, a and x in stack frame (*M[a+i*4]:=x)*

Name	*Effect*	*Trees*
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	+
MUL	$r_i \leftarrow r_j \times r_k$	*
SUB	$r_i \leftarrow r_j - r_k$	-
DIV	$r_i \leftarrow r_j / r_k$	/
ADDI	$r_i \leftarrow r_j + c$	+ CONST, + CONST, CONST
SUBI	$r_i \leftarrow r_j - c$	- CONST
LOAD	$r_i \leftarrow M[r_j + c]$	MEM + CONST, MEM + CONST, MEM CONST, MEM
STORE	$M[r_j + c] \leftarrow r_i$	MOVE MEM + CONST CONST, MOVE MEM + CONST CONST, MOVE MEM CONST, MOVE MEM
MOVEM	$M[r_j] \leftarrow M[r_i]$	MOVE MEM MEM

LOAD $r_1 \leftarrow M[fp+a]$
ADDI $r_2 \leftarrow r_0 + 4$
MUL $r_2 \leftarrow r_i * r_2$
ADD $r_1 \leftarrow r_1 + r_2$
ADD $r_2 \leftarrow fp + x$

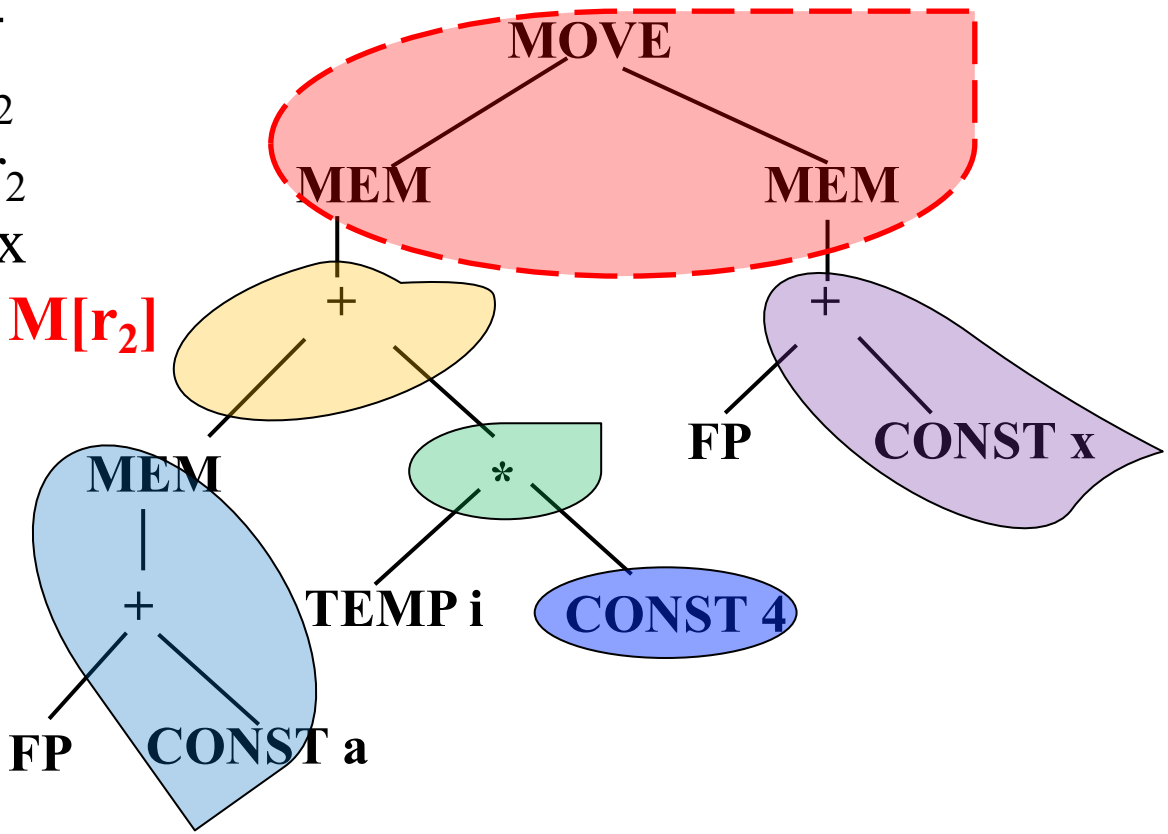


Example: Instruction Emission

- **a[i] := x**, assuming i in register, a and x in stack frame (*M[a+i*4]:=x*)

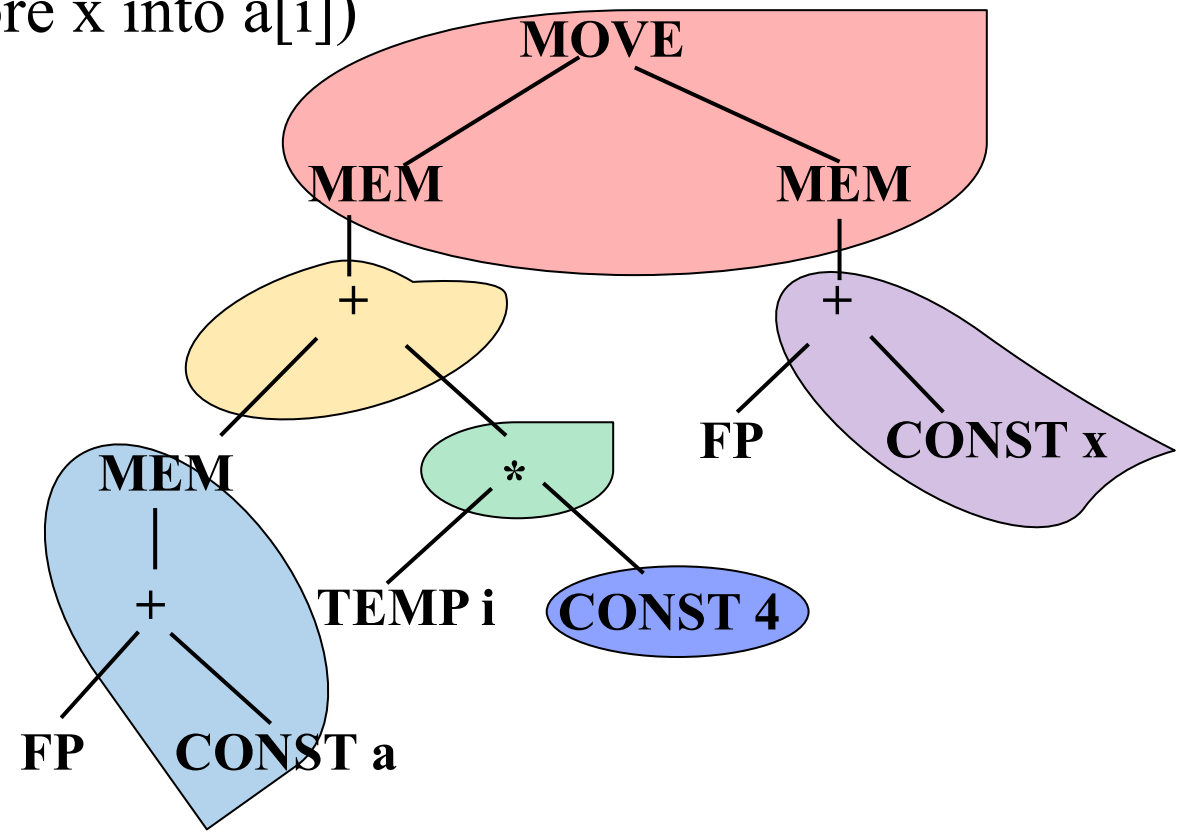
Name	*Effect*	*Trees*
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	+
MUL	$r_i \leftarrow r_j \times r_k$	*
SUB	$r_i \leftarrow r_j - r_k$	-
DIV	$r_i \leftarrow r_j / r_k$	/
ADDI	$r_i \leftarrow r_j + c$	+ CONST, + CONST, CONST
SUBI	$r_i \leftarrow r_j - c$	- CONST
LOAD	$r_i \leftarrow M[r_j + c]$	MEM + CONST, MEM + CONST, MEM CONST, MEM
STORE	$M[r_j + c] \leftarrow r_i$	MOVE MEM + CONST CONST, MOVE MEM + CONST, MOVE MEM CONST, MOVE MEM
MOVEM	$M[r_j] \leftarrow M[r_i]$	MOVE MEM MEM

LOAD $r_1 \leftarrow M[fp+a]$
ADDI $r_2 \leftarrow r_0 + 4$
MUL $r_2 \leftarrow r_i * r_2$
ADD $r_1 \leftarrow r_1 + r_2$
ADD $r_2 \leftarrow fp + x$
MOVEM $M[r_1] \leftarrow M[r_2]$



Example: Instruction Emission

$r_1 = M[fp+a]$ (Load array base address)
$r_2 = r_0 + 4$ (Load element size constant)
$r_2 = r_i * r_2$ (Calculate i*4 for indexing)
$r_1 = r_1 + r_2$ (Calculate final array element address)
$r_2 = fp + x$ (Load value of variable x)
$M[r_1] = M[r_2]$ (Store x into a[i])



Summary of Maximal Munch

- The algorithm is greedy - it always picks the largest possible pattern at each point
- If two tiles of equal size match, the choice is arbitrary (or based on cost)
- Instructions are generated in reverse order (child subtrees first, then parent)

2. Algorithms for Instruction Selection

- Maximal Munch
- Dynamic Programming
- Tree Grammar

Dynamic Programming: Overview

- Maximal Munch finds *optimal* but not always *optimum*
- DP finds the true **optimum** — minimum total cost
- Works **bottom-up** (opposite of Maximal Munch)

Core idea: Assign a cost to every node = the minimum total instruction cost for the subtree rooted at that node.

Dynamic Programming: Overview

- **Goal**: find minimum total cost tiling of tree
- **Idea**: avoid unnecessary re-computation of subtree costs
- Cost assigned to every node in IR tree
  - For each tile t of cost c_t that matches at node n, cost of matching t is:

$$c_t + \sum_{all\ leaves\ i\ of\ t} c_i$$

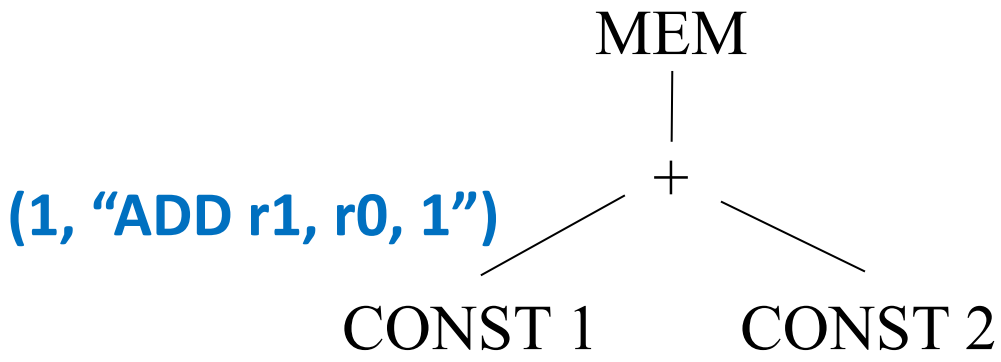
- Find cost of best instruction sequence that can tile subtree rooted at node.

Dynamic Programing Steps

- **Algorithm works bottom-up** (Cost of each subtree has already been computed)
  1. Process tree **bottom-up** (leaves first)
  2. At each node *n*, try **every tile** that matches
  3. For tile *t* with cost *c* and leaves s_1, s_2, ...:
     total cost = $c + \Sigma$ cost(s_i)
  4. Choose the tile with **minimum total cost**
  5. Record the chosen tile at node *n*
- Then emit instructions top-down by following the recorded tiles.

Example: Dynamic Programming

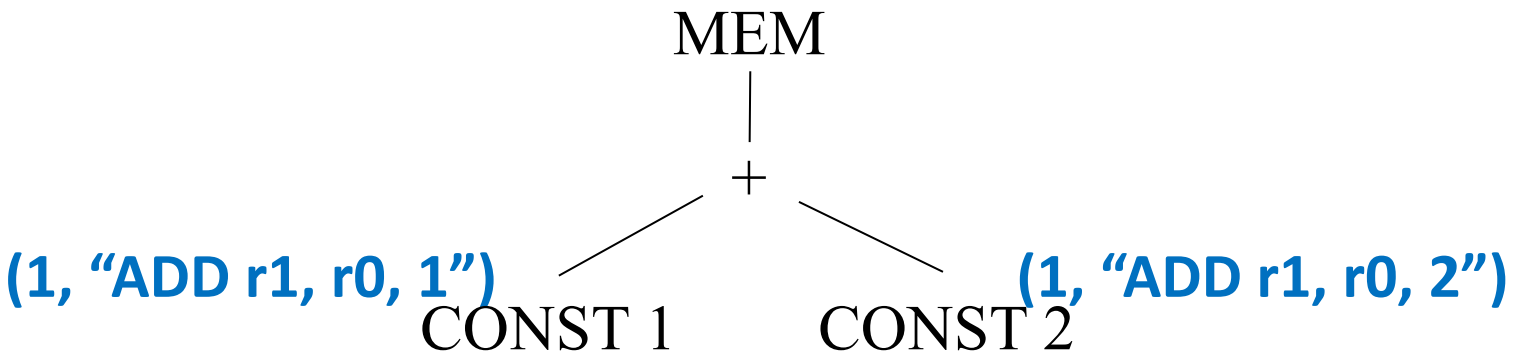
- Consider MEM(PLUS(CONST(1), CONST(2)))



For each node, we compute the (a, b) pair, where

- a is the minimum cost of a node
- b is the optimal instruction

Example: Dynamic Programming

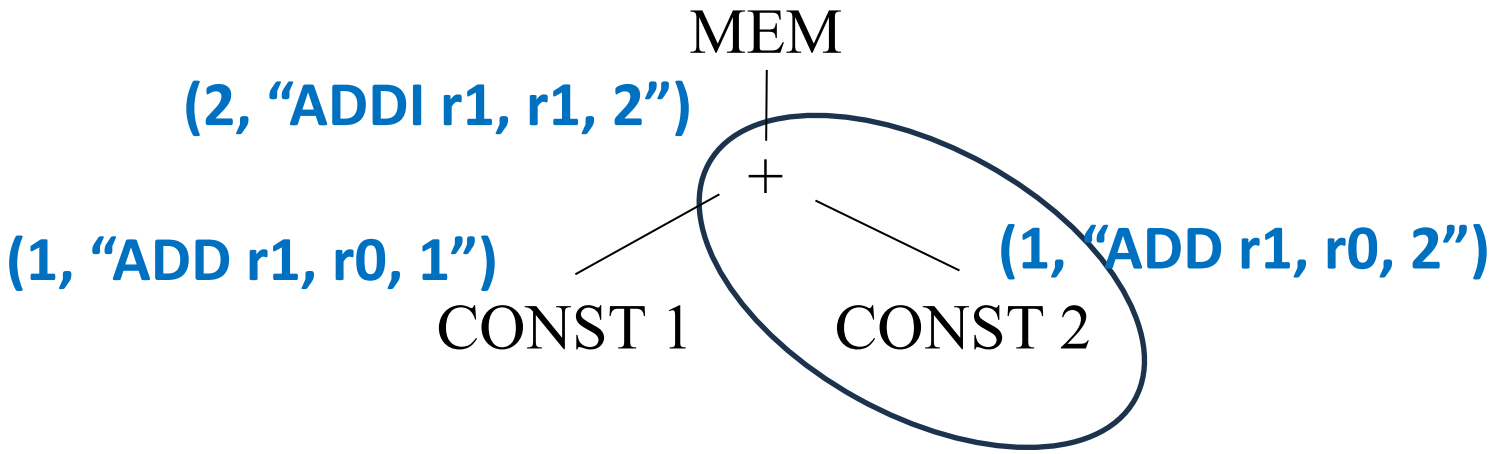


1. CONST 1 and CONST 2 nodes

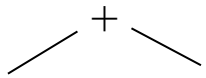
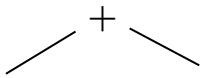
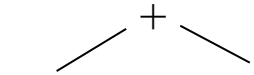
- The only tile that matches CONST 1 is an ADDI instruction with cost 1(similar for CONST2)

Pattern (Tile)	Instruction Cost	Leaves Cost	Total
CONST	1(ADDI)	0	1

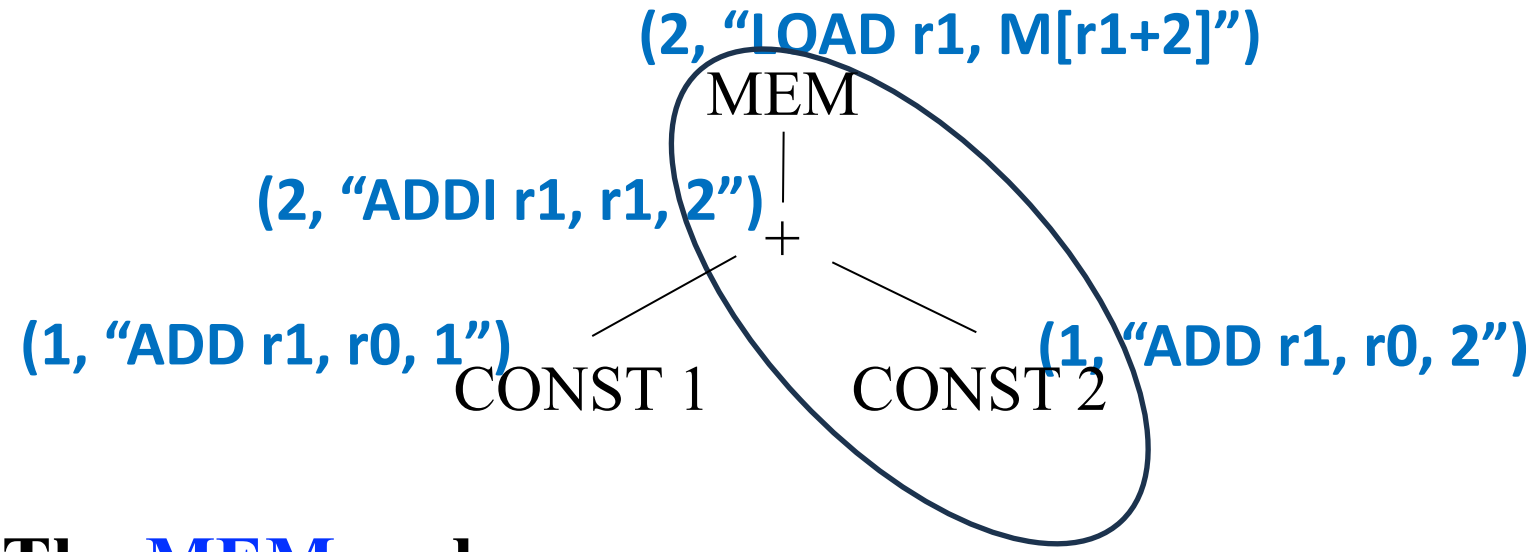
Example: Dynamic Programming



2. The + node: Several tiles match the node

Tile	Instruction	Tile Cost	Leaves Cost	Total cost
	ADD	1	1+1	**3**
	ADDI	1	1	**2**
	ADDI	1	1	**2**

Example: Dynamic Programming



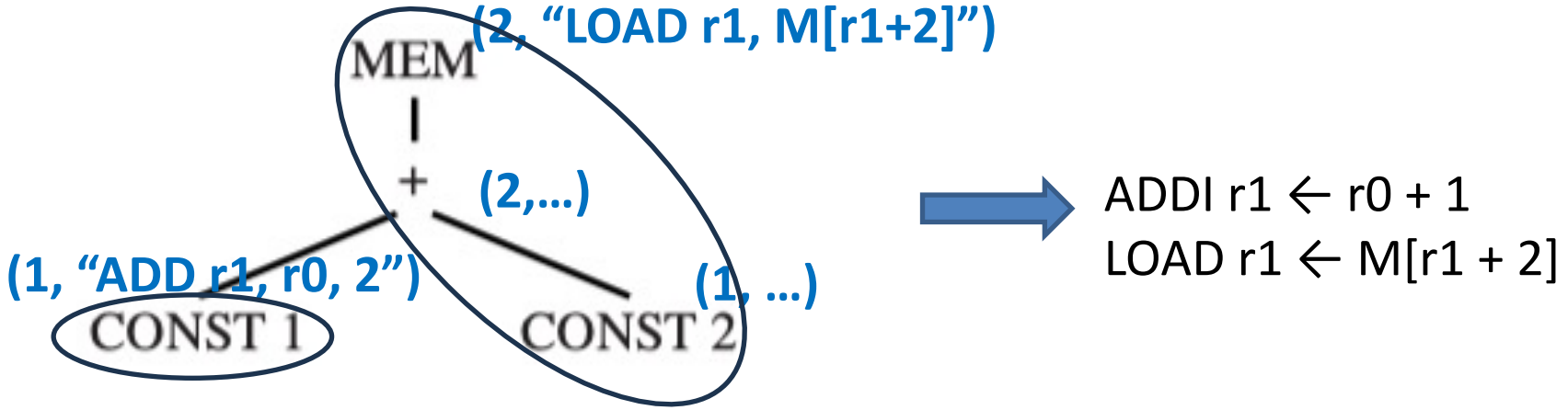
3. The MEM node

Tile	Instruction	Tile Cost	Leaves Cost	Total cost
MEM |	LOAD	1	2	**3**
MEM + CONST	LOAD	1	1	**2**
MEM + CONST	LOAD	1	1	**2**

Dynamic Programming: Instruction Emission

- Once the cost of the root node (and thus the entire tree) is found, the instruction emission begins.

```
function Emission(node n):
    Foreach leaf l_i of the tile selected at node n:
        Emission(l_i) // Recursive call on each uncovered child
    Then emit the instruction matched at node n
```



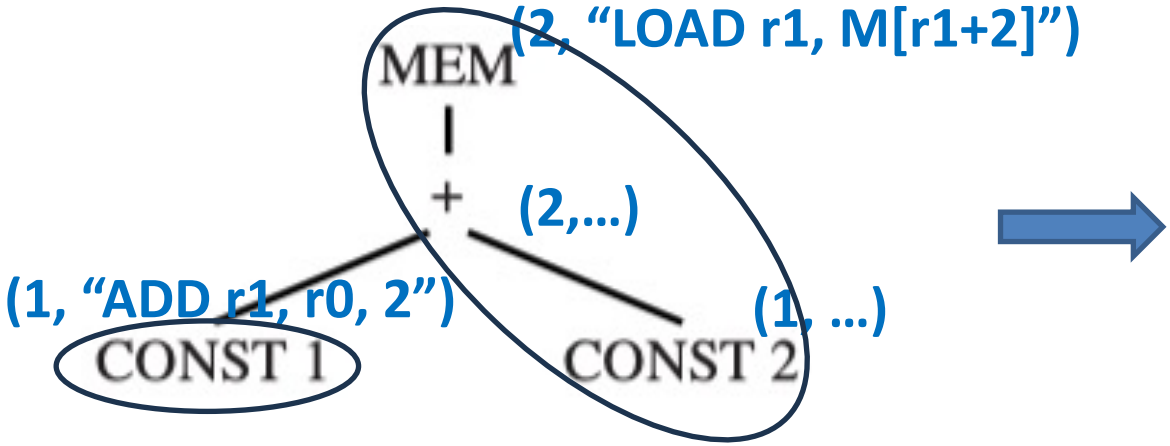
Example: Instruction Emission

1. Start at root: Call Emission(MEM)

– The selected tile at MEM has one leaf: CONST 1 node

→ Call Emission(CONST 1) first

function Emission(node n):
  Foreach leaf l_i of the tile selected at node n:
    Emission(l_i) // Recursive call on each uncovered child
  Then emit the instruction matched at node n



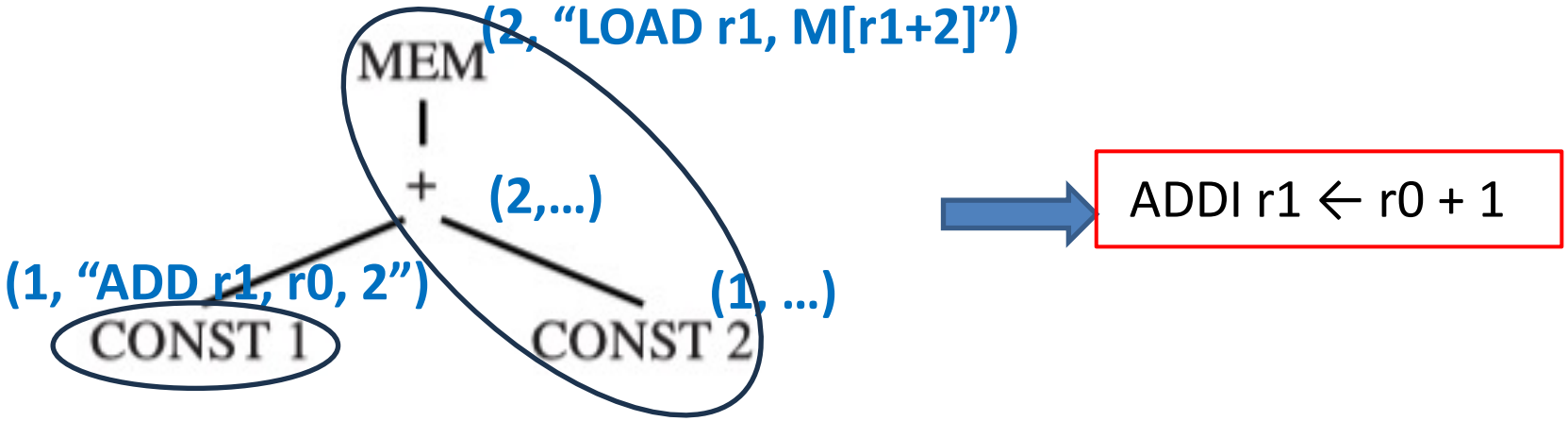
Example: Instruction Emission

2. Process CONST 1 node: Call Emission(CONST 1)

– The CONST 1 tile has no leaves

→ Emit instruction, and return to Emission(MEM)

function Emission(node n):
  Foreach leaf l_i of the tile selected at node n:
    Emission(l_i) // Recursive call on each uncovered child
  Then emit the instruction matched at node n



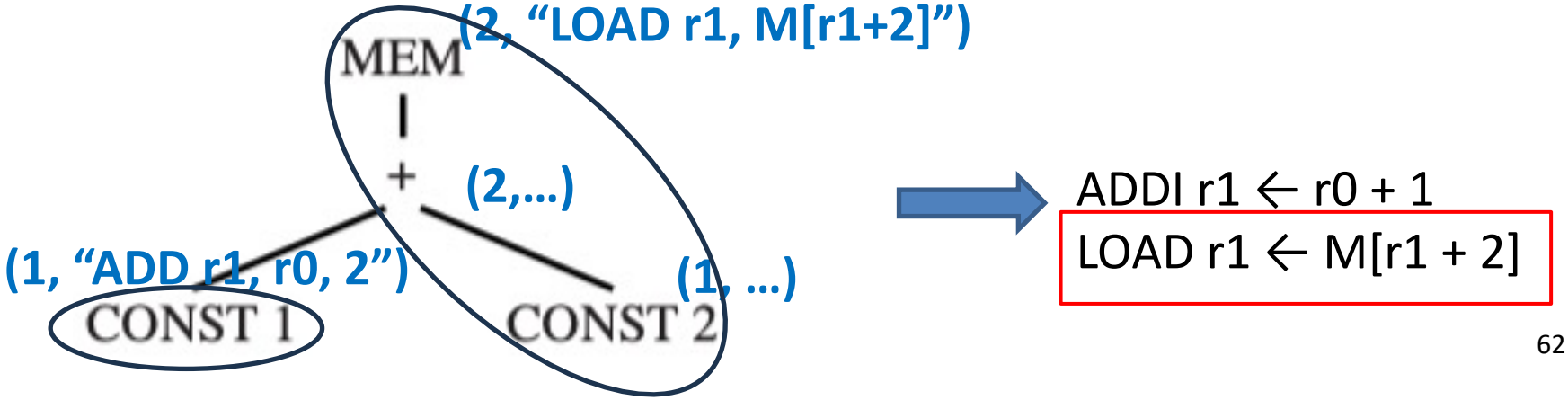
Example: Instruction Emission

3. Complete MEM node: Back to Emission(MEM)

– Already processed CONST 1 leaf

→ Emit instruction LOAD r1 ← M[r1 + 2]

function Emission(node n):
  Foreach leaf l_i of the tile selected at node n:
    Emission(l_i) // Recursive call on each uncovered child
  Then emit the instruction matched at node n



Efficiency of Tilling Algorithms

Parameter	Meaning
T	Number of different tiles
K	Average nodes per matching tile
K'	Largest number of nodes to examine (size of largest tile)
T'	Average number of tiles matching at each node
N	Number of nodes in input tree

- **Maximal munch**: proportional to (K' + T') N / K
- **Dynamic programming**: proportional to (K' + T') N
- K, K' and T' are constant, the running time of all of these algorithms are linear.

2. Algorithms for Instruction Selection

- Maximal Munch
- Dynamic Programming
- Tree Grammar

Motivation

- **Problem:** For machines with complex instruction sets and several classes of registers and addressing modes
  - Hard to use the simple tree patterns and tiling alg.

- **Goal:** We want "instruction selector generators!"
  - Define tiles in a separate specification
  - Use a generic tree pattern matching algorithm to compute tiling

Instruction Selection via Tree Grammars

- **Goal:** instruction selector generators?
- **Solution**
  1. Use a **tree grammar** (a special context-free grammar) to describe the tiles,
  2. Reduce instruction selection **to** a **parsing problem**!
  3. Use a generalization of the dynamic programming algorithm for the "parsing"

Instruction Selection via Tree Grammars

- Each nonterminal represents a register class or storage location.
- The relationships for tiles are encoded as **rewriting rules**. Each rule comprises of
  - A production in a tree grammar
  - An associated cost
  - A code generation template

Pattern, replacement	*Cost*	*Template*
$+(reg_1,reg_2) \rightarrow reg_2$	1	add r1, r2
$store(reg_1, load(reg_2)) \rightarrow done$	5	movem r2, r1

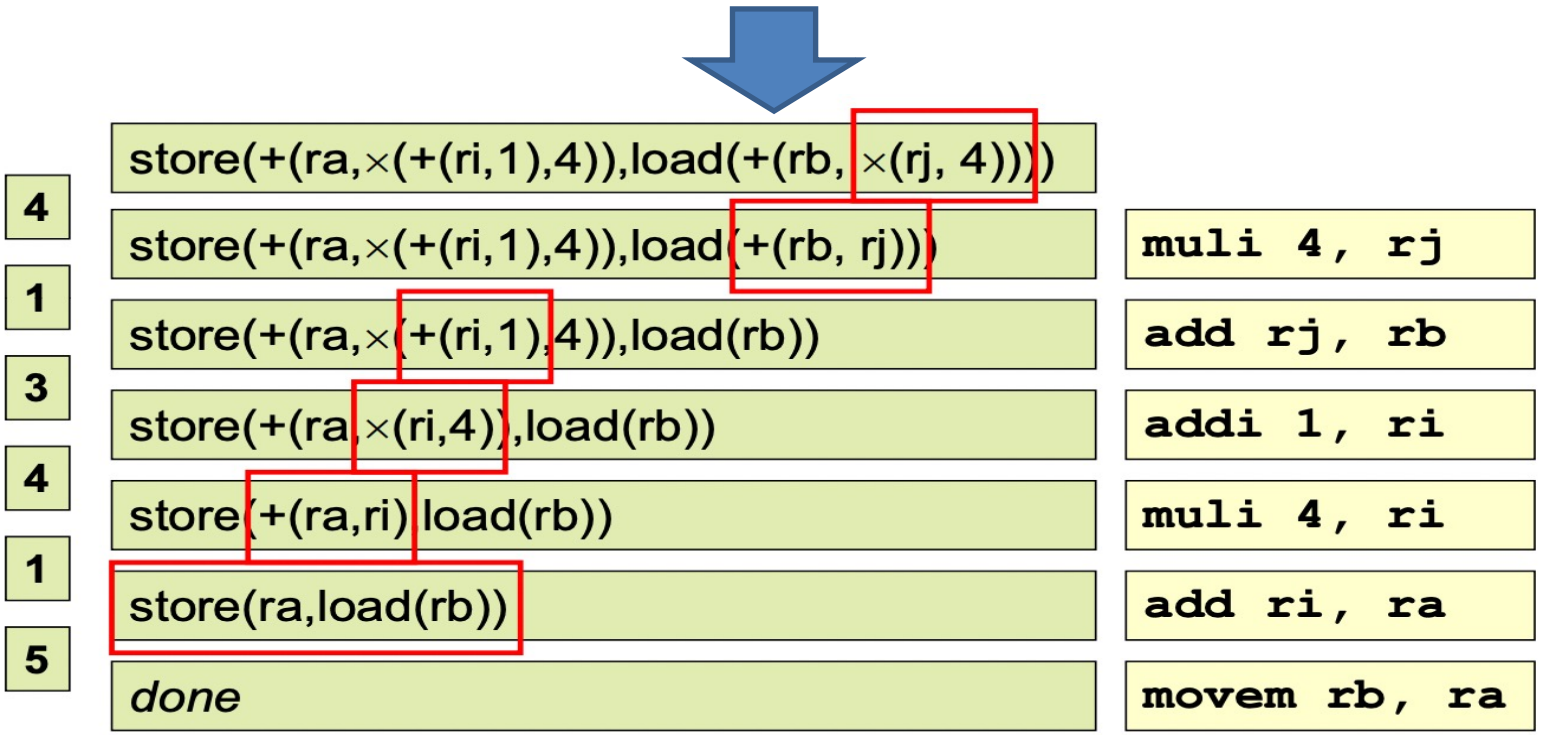
Instruction Selection via Tree Grammars

- **The tree grammars can be ambiguous**
  - There are many different instruction sequences implementing the same expression.

- **Which "parsing algorithms" to use?**
  - The parsing techniques described in Chapter 3 are not very useful
  - However, a generalization of the dynamic-programming algorithm works quite well

Example: Instruction Selection as "Parsing"

#	*Pattern, replacement*	*Cost*	*Template*
1	$+(reg_1,reg_2) \rightarrow reg_2$	1	`add r1, r2`
2	$\times(reg_1,reg_2) \rightarrow reg_2$	10	`mul r1, r2`
3	$+(num,reg_1) \rightarrow reg_2$	1	`addi num, r1`
4	$\times(num,reg_1) \rightarrow reg_2$	10	`muli num, r1`
5	$store(reg_1, load(reg_2)) \rightarrow$ *done*	5	`movem r2, r1`



Recap and Summary

- Several compilation tasks can be formally described and automated via formal grammars

Compiler Task	**Description Formulation**
Lexical analysis	Regular expressions
Syntax analysis	LL, LR grammars
Instruction selection	Regular tree grammars

- Several tools have been developed to generate code from tree grammar specifications:

Compiler Task	**Approach**
Twig	Bottom-up dynamic programming
BURG	Bottom-up rewriter grammar
LLVM TableGen	Pattern matching tablegen

3. CISC Machines

Algorithms for Instruction Selection

- For **RISC**, there is usually no difference at all between optimum and optimal tilings, dur to small, uniform costs.
- For **CISC**, the difference between optimum and optimal tilings is noticeable (due to complex instructions that combine multiple operations.)
- Thus, for RISC, the simpler tiling algorithms suffice

RISC vs. CISC

RISC machine	**CISC machine**
32 registers	few registers (16, or 8, or 6)
one class of integer/pointer register	registers divided into different classes; some operations available only on certain registers;
arithmetic operations only between registers	arithmetic operations can access registers or memory through "addressing modes";
simple addressing modes: load and store instructions with only the M[reg+const] addressing mode	several different addressing modes
“three-address” instructions of the form **r1 ← r2 ⊕ r3**	"two-address" instructions of the form **r1 ← r1 ⊕ r2**;
every instruction exactly 32 bits long	variable-length instructions
one result or effect per instruction	instructions with side effects such as "autoincrement" addressing modes.

Handling CISC Challenges

Challenge	Solution	Example
Few registers	Generate TEMP nodes freely; rely on register allocator	Let register allocator decide when to spill
Register classes	Use move instructions to transfer values between registers	MOVEADDR eax ← esi (move address register to general)
Two-address instructions	Add extra move instructions, rely on register allocator to optimize	mov ecx, eax add ecx, ebx (for eax + ebx when preserving eax)
Memory operands	Use specialized tiles for memory operands	add eax, [ebx+8] (direct memory operand)
Complex addressing modes	Create specific tiles for common address calculations	mov eax, [ebx+4*ecx+8] (scaled index addressing)

Problem of CISC: Few registers

- **CISC machines have few registers**

- **Solution**: generate TEMP nodes freely, and assume that the register allocator will do a good job

Problem of CISC : Register Classes

- **Classes of registers**: registers are divided into different classes; some operations available only on certain registers
  - E.g., Pentium的乘法指令要求将左操作数放入寄存器eax，结果的高位放入rdx
- **Solution**: Use move instructions to transfer values between registers
  - 将操作数和结果显式地传送到相应的寄存器,
    E.g., for t1 ← t2 × t3:

```
mov eax, t2                    ; eax ← t2
mul t3                         ; eax ← eax × t3; edx ← garbage
mov t1, eax                    ; t1 ← eax
```

Problem of CISC : Two-Address Instructions

- **Two-address instructions:** r1 ← r1 ⊕ r2

- **Solution**: Add extra move instructions, and rely on the register allocator to optimize
  - Example: In order to implement t1 ← t2 + t3

```
mov t1, t2      t1 ← t2
add t1, t3      t1 ← t1 + t3
```

如果寄存器分配算法可以把t1和t2分配到同一个寄存器，那么就可以删除这条传送指令

Problem of CISC: Memory Operands

- **Arithmetic operations can address memory**

- **Solution**: use specialized tiles for memory operands
  - Fetch all the operands into registers before operating and store them back to memory afterwards.
  - E.g.,: these two sequences compute the same thing:

```
mov eax, [ebp - 8]
add eax, ecx
mov [ebp - 8], eax
```

```
add [ebp - 8], ecx
```

Problem of CISC : Several Addressing Modes

- **Several addressing modes have t**wo advantages
  - They “trash” fewer registers
  - Shorter instruction encoding

- Solution: Create specific tiles for common address calculations

Problem of CISC: Instruction with Side Effects

- **Instructions with side effects:**
  - Problem: some machines have an "autoincrement" memory fetch instruction whose effect is:

$$r2 \leftarrow M[r1]; r1 \leftarrow r1 + 4$$

  - Difficult to model using tree patterns, as it produces two results.
  - There are three solutions:
    - Ignore the autoincrement instructions, and hope they go away.
    - Try to match special idioms in an ad hoc way, within the context of a tree pattern-matching code generator.
    - Use a different instruction algorithm entirely, one based on DAG patterns instead of tree patterns.

Summary

- Tiling algorithms find optimal/optimum instruction sequences:
  - **Maximal Munch:** Simple, top-down, produces optimal tilings
  - **Dynamic Programming:** Bottom-up, produces optimum tilings
- Emission process is critical in both algorithms:
  - Maximal Munch emits during pattern matching, in reverse visit order
  - Dynamic Programming does a separate emission phase after finding costs

Thank you all for your attention